

Curated File Guide: server.mjs

The local backend that saves drafts, runs compilers, checks GitHub publishing access, imports/pushes site files, handles updates, and relays AI.

Before The Code: File Overview

Abstract

server.mjs is the local nervous system of the OMB Portfolio Builder. It is not the public website and it is not the Electron shell. It is the private backend that runs on the owner's machine and gives the builder the powers a browser page should not have by itself: reading and writing local files, saving drafts, accepting uploads, running compilers, using Git, checking publishing authorization, importing a target site, launching installers, and relaying AI requests. The file is intentionally broad because the builder is local-first. Instead of depending on a hosted database and a remote application server, the desktop app starts this Node server beside the user-owned workspace and talks to it through local API endpoints.

The most important idea is boundary control. The frontend can draw buttons and collect input, but it cannot safely decide where files are written, which Git repository receives a push, whether a command should run, or whether a request came from the local computer. server.mjs owns those decisions. It is the layer that turns a UI action into a guarded filesystem, compiler, Git, installer, or AI operation.

Introduction

When the builder starts, Electron loads a local page. That page needs catalog data, templates, upload handling, compile actions, publishing actions, and update checks. A static HTML page cannot perform those operations directly without either exposing dangerous privileges or running into browser security walls. server.mjs solves that by becoming a small local application server. It serves the builder files, exposes API endpoints under /api, and keeps privileged work in Node where the filesystem, process runner, and Git commands are available.

A useful mental model is to treat this file like a workshop manager. The frontend is the workbench surface where the user sees projects, editors, buttons, and dialogs. server.mjs is the person behind the counter who knows where the tools are, where files belong, whether a customer has permission to ship something, and whether the requested operation is safe. If the frontend says 'save this PDF under this project,' server.mjs sanitizes the path and writes it under the correct docs folder. If the frontend says 'compile this C++ program,' server.mjs finds g++, creates a temporary run folder, captures terminal output, and returns a result object. If the frontend says 'apply to site,' server.mjs checks Git authorization first, then synchronizes publishable files and pushes them.

Background Information

The file uses built-in Node modules rather than a large web framework. createServer from node:http accepts requests, node:fs/promises handles files, node:child_process runs compilers and Git, node:path normalizes paths, node:crypto hashes authorization cache scopes, and node:os identifies the current machine. This keeps the backend predictable and package-light. The tradeoff is that routing and request handling are written manually. That is why handleApi is large: it plays the role that Express or another framework would normally play.

There are three important roots. root is the builder workspace that contains the site files and local draft catalog. portfolioRoot is the publish mirror, which may be separate from root so generated website files can be pushed without mixing in builder-only files. compileRoot is where project-local code workspaces and temporary compiler run folders live. Most path-related functions exist to make sure a request stays inside one of these roots.

The backend is also intentionally local-sensitive. Some endpoints are read-only, but write endpoints reject non-local callers. That is why functions such as `isLocalRequest` matter: the builder may be opened in a browser or accessed on a LAN during testing, but operations that change files, compile code, install tools, or publish should be performed only from the owner's computer.

Purpose Of The File

The purpose of `server.mjs` is to make the builder operational instead of decorative. Without it, `template-preview.html` and `template-preview.js` would still be able to render a screen, but they would not be able to reliably save drafts, upload project evidence, run compilers, authenticate GitHub targets, import an existing website, launch updates, or call private AI providers. The file therefore holds the application's real side effects.

The file is also a protection layer. It validates filenames, normalizes repository URLs, keeps publish authentication scoped to a machine and target, prevents stageable publishing paths from wandering outside the publish mirror, blocks remote write requests, caps text fetched from external sources, and strips terminal output into UI-safe strings. Those decisions are not cosmetic. They prevent the portfolio builder from becoming a general-purpose remote command runner or a tool that accidentally pushes the wrong files.

Primary Responsibilities

The first responsibility is serving local files. The same server that handles `/api` requests also serves `index.html`, `styles.css`, `script.js`, `template-preview.html`, project catalogs, assets, and uploaded docs during local preview. The static-serving logic checks that the requested path stays inside the workspace before reading the file. That simple check is what prevents a URL from escaping into arbitrary folders on the machine.

The second responsibility is compilation. `Compile Code` works because the backend can write source files to a project workspace, discover tools such as `node`, `python`, `gcc`, `g++`, `javac`, `iverilog`, `vvp`, and `LTspice`, run those tools with timeouts, collect stdout and stderr, cache successful builds, and return terminal output immediately to the frontend. The backend treats each language differently because JavaScript, Python, C, C++, Java, Verilog, SystemVerilog, LTspice, and HTML do not build the same way.

The third responsibility is publishing. Publishing is not just 'write `projects.json`.' The backend must verify that the publish mirror is a Git repository, know its origin remote and branch, check compatibility with a static portfolio site, synchronize with the remote to avoid non-fast-forward errors, stage only approved publish paths, commit if there are changes, and push to the selected branch. This is why the publishing code is careful and verbose.

The fourth responsibility is intelligence. For local AI, the backend can use an OpenAI API key or a local Ollama model. It also enriches portfolio context by reading public project links, same-site files, GitHub repositories, GitHub profiles, README files, and selected source files when the question calls for it. The browser cannot safely hold private API keys, so `server.mjs` keeps that path private.

The fifth responsibility is application maintenance. The update functions check GitHub Releases, download an installer, write a PowerShell handoff script, close the running app, run the installer, and relaunch the updated executable. This is backend work because the browser cannot replace the running desktop app.

Important Data Objects And Why They Matter

`compileLanguageProfiles` is the backend's language map. It tells the compiler system what default filename to use, which file extensions belong to each language, what human label to show, which tools are required, and which Winget package can install the tool if automatic installation exists. Without this object, `compileAndRunCode` would need hardcoded language assumptions scattered throughout the file.

`compileToolCandidates` is the tool-discovery map. It lists command names and likely Windows install paths for Git-adjacent and compiler-adjacent executables. `findTool` uses this object so the app can work even when a tool

is installed but not perfectly exposed on PATH. The caches `compileToolCache` and `compileToolVersionCache` exist so repeated compiles do not waste time rediscovering the same executables.

`publishPaths` is the publishing allowlist. It lists the site files and folders the backend is allowed to copy into the publish mirror and stage in Git. This is one of the most important safety objects in the file. Without an allowlist, an Apply to site operation could accidentally stage builder-only files, private drafts, temporary compile output, or unrelated local files.

`portfolioAiInstructions` is the system behavior contract for the portfolio assistant. It tells the model how to distinguish general conversation, general engineering, general knowledge, and portfolio-specific questions. It also tells the model when to use context, when not to invent, and how to handle public GitHub source excerpts. That string is long because it is the policy boundary for the AI feature.

Top-Level Objects And Variables

This section explains the long-lived objects and variables before the function walkthrough. These are the values that give the file memory, configuration, API handles, DOM anchors, path boundaries, cache state, and behavior maps. They are explained by meaning, not by line number.

Filesystem Location

root

root is the absolute folder that contains server.mjs. Every local static-file lookup starts from this folder unless the app is deliberately publishing or compiling somewhere else. It is calculated from import.meta.url instead of the current shell directory so the backend behaves the same whether Electron, PowerShell, or another launcher starts it. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include installCompilerTools, resolveInsideRoot, getPublishTargetInfo, runGit, runOptionalCommand, writeTargetCustomDomain, publishAuthCacheScope, syncFromPublishTarget, syncPortfolioPublishFiles, handleApi.

portfolioRoot

portfolioRoot is the folder treated as the publishable website mirror. It normally equals root, but OMB_PORTFOLIO_WORKSPACE can redirect it to a separate portfolio folder. Publishing functions use this boundary when copying and staging public website files. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include resolveInsidePortfolioRoot, getPublishTargetInfo, runPublishGit, runGitWithInput, storeGitCredentials, writeTargetCustomDomain, ensureGitRepository, publishAuthCacheScope, authenticateGitHubForTarget, syncPortfolioPublishFiles.

compileRoot

compileRoot is the separate workspace for Compile Code projects. It is deliberately outside the public site folder so temporary source files, binaries, VCD waveforms, and compiler output do not get mixed into the portfolio website. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include runProcess, compileToolStatus, resolveInsideCompileRoot.

draftPath

draftPath is the private local draft catalog path. It stores builder-side edits that have not necessarily been applied to the public projects.json file. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include syncFromPublishTarget, handleApi.

catalogPath

catalogPath is the public portfolio catalog path. The website runtime reads this file to render the public project list, profile, sections, fun facts, links, and rich content. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include handleApi.

publishAuthCachePath

publishAuthCachePath is the local file that remembers recent publishing authorization. It is intentionally local to the machine and target so GitHub verification does not have to repeat on every Apply to site click. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include readPublishAuthCache, writePublishAuthCache.

publishPaths

publishPaths is the allowlist of files and folders that can be copied and staged during publishing. This is a critical safety object: it prevents builder-only files, temp compiler output, credentials, and internal docs from being pushed as website content. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include syncPortfolioPublishFiles, publishSiteChanges.

Temporary Calculation

port

port is the TCP port used by the local HTTP server. PORT can override it for development or packaging, while the default supports the local builder preview without requiring a user to choose a port. Functions that reference it include handleApi.

host

host is the network interface binding for the server. The default allows the local runtime to listen broadly, but write operations still rely on isLocalRequest so privilege is not granted merely because a browser can reach the server. Functions that reference it include parseGitHubSourceUrl, callOllamaPortfolioAi, storeGitCredentials, handleApi.

execFileAsync

execFileAsync is the Promise-based wrapper around Node's execFile. It lets Git, version checks, and other short commands use async/await instead of nested callbacks. Functions that reference it include findTool, runGit, runOptionalCommand.

packageJson

packageJson holds metadata from package.json, especially the current builder version used by update checks, release comparisons, and status messages. Functions that reference it include getGitStatus, getUpdateInfo.

types

types maps file extensions to HTTP Content-Type headers. Static serving uses it so browsers interpret CSS as CSS, JavaScript as JavaScript, PDFs as PDFs, icons as icons, and uploaded assets as the right media type. Visible object fields include .css, .csv, .html, .ico, .js, .json, .md, .pdf, .png, .jpg, .jpeg, .svg, .txt, .webp, .xml, .xlsx, .zip. Those fields form the contract consumed by related functions. Functions that reference it include handleApi.

defaultSiteRepository

defaultSiteRepository stores the optional repository configured through the environment. It gives packaged or developer-launched builds a default target without hardcoding one into the code. Functions that reference it include getPublishTargetInfo, assertPublishAccess.

Runtime State Or Cache

publishAuthCacheTtlMs

publishAuthCacheTtlMs is the normal one-day authorization window. It implements the rule that GitHub publishing should not ask again every time the app opens, but also should not trust an old session forever. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache, publishAuthCacheExpiresAt.

publishAuthHistoryWindowMs

publishAuthHistoryWindowMs is the seven-day window used to count successful authorizations. The cache logic uses it to decide whether the user qualifies for extended trust. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache.

compileToolCache

compileToolCache remembers the resolved path for compiler tools. It prevents every compile click from scanning the filesystem again. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include findTool, installCompilerTools.

compileToolVersionCache

compileToolVersionCache remembers version strings for tools after they have been checked. This makes status panels faster and keeps repeated diagnostics from slowing the UI. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include toolVersionLine.

Git Publishing And Authorization State

publishAuthExtendedTtlMs

publishAuthExtendedTtlMs is the thirty-day extended authorization window. It is used after repeated successful authorizations prove the same device and target are consistently owned by the same publisher. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache.

publishAuthExtendedThreshold

publishAuthExtendedThreshold is the number of successful authorizations required before the builder extends trust. It turns the user's requested security rule into a concrete number. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache.

publishAuthorizationHelp

publishAuthorizationHelp is the human-readable troubleshooting text shown when publishing is blocked. It explains the sequence the user should follow: configure target, authenticate, load if needed, then apply. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include publishAccessError, handleApi.

gitCandidates

gitCandidates lists command names and likely Windows Git paths. Git operations use this list to find Git even when PATH is incomplete, which is common on Windows installs. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include runGit, runGitWithInput.

githubTextFilePattern

githubTextFilePattern tells the AI source reader which GitHub files are worth fetching as text. It includes code, README, HDL, schematic-like text formats, scripts, and engineering project files. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include sourceUrlAllowed, scoreGitHubFile, fetchGitHubRepositorySource.

githubSkipFilePattern

githubSkipFilePattern tells the AI source reader which GitHub files to skip because they are binary, huge, media-oriented, or not useful as prompt text. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include sourceUrlAllowed, fetchGitHubRepositorySource.

Lookup Collection

blockedAppUpdateVersions

blockedAppUpdateVersions is a Set of release versions the updater must skip. It exists so a known bad release can be blocked without deleting every release artifact from GitHub. Functions that reference it include getUpdateInfo.

Portfolio Data State

compileLanguageProfiles

compileLanguageProfiles is the language rulebook for the IDE feature. Each language entry defines the default filename, valid extensions, display label, required tools, and installer hints. Compile, build, run, simulate, and beautify operations all depend on this map. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Visible object fields include c, defaultFile, extensions, label, primaryTools, winget, cpp, verilog, systemverilog, ltspice, java, javascript, python, html. Those fields form the contract consumed by related functions. Functions that reference it include normalizeCodeLanguage, languageFromFileName, safeCodeFileName, compileToolStatus, hdlFilesFromPayload, compileWorkspaceFilesFromPayload, compileAndRunCode, installCompilerTools.

Compiler Or Language Configuration

compileToolCandidates

compileToolCandidates maps compiler tool names to possible executable locations. It lets the app find gcc, g++, javac, java, iverilog, vvp, python, node, and LTspice even when a normal terminal cannot. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Visible object fields include gcc, g++, clang, clang++, cl, javac, java, node, python, iverilog, vvp, ltspice. Those fields form the contract consumed by related functions. Functions that reference it include findTool.

Ai Configuration Or Conversation State

portfolioAiInstructions

portfolioAiInstructions is the system prompt for the portfolio assistant. It defines how the model should distinguish greetings, general electronics questions, portfolio-specific questions, public GitHub code requests, and uncertain answers. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include callOllamaPortfolioAi, handlePortfolioAi.

Code Walkthrough

This walkthrough is function-by-function. Related ideas are mentioned inside each function's explanation, but the headings are the actual function names so the document can be used as a code-reading companion even as line numbers change.

HTTP API surface

The functions in this group all support http api surface. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function securityHeaders(extra = {})

Purpose and intuition: securityHeaders belongs to http api surface. This function belongs to the local HTTP/API layer. It either shapes responses, reads requests, or decides whether a request is allowed to perform local work. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: extra is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means securityHeaders performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without securityHeaders, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function sendJson(response, status, data)

Purpose and intuition: sendJson belongs to http api surface. This function belongs to the local HTTP/API layer. It either shapes responses, reads requests, or decides whether a request is allowed to perform local work. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: response is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. status is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. data is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: serializes structured data before sending or saving it; writes the HTTP response directly.

What it calls or works with: It works with securityHeaders. Those calls show that sendJson is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without sendJson, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function isLocalRequest(request)

Purpose and intuition: isLocalRequest is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Parameters and data it receives: request is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `isLocalRequest` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `isLocalRequest`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

address: `address` stores a computed value for temporary calculation. It is initialized from `request.socket.remoteAddress || ""`. Within the function it is returned to the caller; assigned or updated later in the function.

async function readRequestJson(request)

Purpose and intuition: `readRequestJson` reads `RequestJson` from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Parameters and data it receives: `request` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

What it does: Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: parses JSON rather than treating incoming text as already trusted data; throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: No other named helper from this file is detected inside the body. That means `readRequestJson` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `readRequestJson`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

chunks: `chunks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; passed into `parse`; assigned or updated later in the function.

size: `size` stores a numeric value for temporary calculation. It is initialized from `0`. Within the function it is assigned or updated later in the function.

chunk: `chunk` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `length`; passed into `await`, `push`.

async function handleApi(request, response, url)

Purpose and intuition: `handleApi` is the backend router. It receives the parsed URL and decides which local API feature should run: project catalog reads, uploads, draft saves, compile actions, compiler installation, target authentication, load from target, apply to site, update checks, security reports, AI calls, or builder download

reports. In a framework such as Express this would be spread across route handlers. Here it is one explicit switch-like controller.

Purpose and intuition: The function protects the boundary between a browser click and local machine power. Before write operations, it checks whether the request is local. It reads request JSON only when needed, calls the specific operation function, and returns a normalized JSON response. Without `handleApi`, the server could still serve files, but the builder would have no organized command surface.

Parameters and data it receives: `request` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. `response` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: serializes structured data before sending or saving it; uses Node path utilities so Windows path separators and relative paths are handled consistently; writes the HTTP response directly; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

What it calls or works with: It works with `readJsonFile`, `sendJson`, `isLocalRequest`, `getPublishTargetInfo`, `getGitStatus`, `getUpdateInfo`, `getSecurityReport`, `compileToolStatus`. Those calls show that `handleApi` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `handleApi`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

catalog: `catalog` stores a resolved filesystem or route value. It is initialized from `await readJsonFile(draftPath)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `categories`, `projects`; passed into `isArray`, `sendJson`, `writeFile`; assigned or updated later in the function.

body: `body` stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`, `fileName`, `language`, `projectId`, `section`; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code, body.fileName))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `installCompilerTools`, `normalizeCodeLanguage`, `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

saved: saved stores the completed result of an awaited operation. It is initialized from `await saveCompileSource(body)`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

result: result stores the completed result of an awaited operation. It is initialized from `await compileAndRunCode(body)`. Within the function it is accesses properties/methods ok; passed into `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

result: result stores the completed result of an awaited operation. It is initialized from `await installCompilerTools(body.language || "")`. Within the function it is accesses properties/methods ok; passed into `sendJson`; assigned or updated later in the function.

install: install stores the completed result of an awaited operation. It is initialized from `await installGitForWindows()`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

update: update stores the completed result of an awaited operation. It is initialized from `await downloadAndLaunchAppUpdate()`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

auth: auth stores the completed result of an awaited operation. It is initialized from `await authenticateGitHubForTarget(body || {})`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `sendJson`; assigned or updated later in the function.

sync: sync stores the completed result of an awaited operation. It is initialized from `await syncFromPublishTarget()`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`, `fileName`, `language`, `projectId`, `section`; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

catalog: catalog stores a computed value for portfolio data state. It is initialized from `body.catalog`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `categories`, `projects`; passed into `isArray`, `sendJson`, `writeFile`; assigned or updated later in the function.

applyingToSite: applyingToSite stores a computed value for temporary calculation. It is initialized from `url.pathname === "/api/apply-catalog"`. Within the function it is assigned or updated later in the function.

publishAccess: publishAccess stores a computed value for Git publishing and authorization state. It is initialized from `null`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `publishSiteChanges`, `sendJson`; assigned or updated later in the function.

target: target stores a resolved filesystem or route value. It is initialized from `applyingToSite ? catalogPath : draftPath`. Within the function it is passed into `sendJson`, `writeFile`; assigned or updated later in the function.

publish: publish stores the completed result of an awaited operation. It is initialized from `await publishSiteChanges(publishAccess)`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`, `fileName`, `language`, `projectId`, `section`; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

projectId: projectId stores a computed value for portfolio data state. It is initialized from `safeSegment(body.projectId, "project")`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `resolveInsideRoot`, `safeSegment`; assigned or updated later in the function.

section: section stores a computed value for portfolio data state. It is initialized from `safeSegment(body.section, "documents")`. Within the function it is passed into `resolveInsideRoot`, `safeSegment`; assigned or updated later in the function.

fileName: fileName stores a computed value for temporary calculation. It is initialized from `safeFileName(body.fileName)`. Within the function it is passed into `normalizeCodeLanguage`, `resolveInsideRoot`, `safeFileName`; assigned or updated later in the function.

base64: base64 stores a computed value for temporary calculation. It is initialized from `String(body.data || "").replace(/^data:[^`. Within the function it is passed into `replace`, `writeFile`; assigned or updated later in the function.

folder: folder stores a computed value for filesystem location. It is initialized from `resolveInsideRoot("docs", projectId, section)`. Within the function it is passed into `mkdir`; assigned or updated later in the function.

filePath: filePath stores a computed value for filesystem location. It is initialized from `resolveInsideRoot("docs", projectId, section, fileName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses

properties/methods startsWith; passed into extname, readFile, sendJson, writeFile; assigned or updated later in the function.

url: url stores a constructed object for network or routing value. It is initialized from new URL(request.url || "/", `http://\${request.headers.host}`). Within the function it is accesses properties/methods pathname; passed into URL, decodeURIComponent, handleApi, sendJson; assigned or updated later in the function.

pathname: pathname stores a computed value for filesystem location. It is initialized from url.pathname === "/" ? "/index.html" : decodeURIComponent(url.pathname). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods startsWith; passed into decodeURIComponent, normalize; assigned or updated later in the function.

filePath: filePath stores a resolved filesystem or route value. It is initialized from path.normalize(path.join(root, pathname)). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods startsWith; passed into extname, readFile, sendJson, writeFile; assigned or updated later in the function.

body: body stores a resolved filesystem or route value. It is initialized from await readFile(filePath). Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into String, authenticateGitHubForTarget, compileAndRunCode, end, installCompilerTools, normalizeCodeLanguage, safeFileName, safeSegment; assigned or updated later in the function.

Backend utility

The functions in this group all support backend utility. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function clampText(value, maxLength = 12000)

Purpose and intuition: clampText belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. maxLength is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means clampText performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without clampText, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function stripHtmlToText(value = "")

Purpose and intuition: stripHtmlToText belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means stripHtmlToText performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without stripHtmlToText, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeCodeLanguage(value = "")

Purpose and intuition: normalizeCodeLanguage standardizes CodeLanguage data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeCodeLanguage performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeCodeLanguage, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[ _s-]+/g, "")`. Within the function it is assigned or updated later in the function.

aliases: aliases stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; assigned or updated later in the function.

function indentBraceCode(code = "")

Purpose and intuition: indentBraceCode belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means indentBraceCode performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without indentBraceCode, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

depth: depth stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into max, repeat; assigned or updated later in the function.

line: line stores a computed value for temporary calculation. It is initialized from rawLine.trim(). Within the function it is accesses properties/methods match; passed into test; assigned or updated later in the function.

formatted: formatted stores a string value for temporary calculation. It is initialized from `\${" ".repeat(depth)}\${line}`. Within the function it is returned to the caller; assigned or updated later in the function.

opens: opens stores a function or callback value for temporary calculation. It is initialized from (line.match(/[{([/g) || []]).length. Within the function it is passed into max; assigned or updated later in the function.

closes: closes stores a function or callback value for temporary calculation. It is initialized from (line.match(/[/\])]/g) || []).length. Within the function it is passed into max; assigned or updated later in the function.

function beautifyCode(code = "", language = "javascript")

Purpose and intuition: beautifyCode belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with normalizeCodeLanguage, indentBraceCode. Those calls show that beautifyCode is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `beautifyCode`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

normalized: `normalized` stores a computed value for temporary calculation. It is initialized from `String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ")`. Within the function it is returned to the caller; passed into `indentBraceCode`; assigned or updated later in the function.

lang: `lang` stores a computed value for temporary calculation. It is initialized from `normalizeCodeLanguage(language)`. Within the function it is assigned or updated later in the function.

async function findExecutableUnder(folder, names = [], maxDepth = 5)

Purpose and intuition: `findExecutableUnder` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `folder` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `names` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `maxDepth` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

What it calls or works with: It works with `pathExists`. Those calls show that `findExecutableUnder` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `findExecutableUnder`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

entries: `entries` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is assigned or updated later in the function.

entry: `entry` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `isDirectory`, `isFile`, `name`; passed into `findExecutableUnder`, `join`.

fullPath: fullPath stores a resolved filesystem or route value. It is initialized from path.join(folder, entry.name). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is returned to the caller; assigned or updated later in the function.

entry: entry starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods isDirectory, isFile, name; passed into findExecutableUnder, join.

found: found stores a resolved filesystem or route value. It is initialized from await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1). Within the function it is returned to the caller; assigned or updated later in the function.

function terminalLine(label, text = "")

Purpose and intuition: terminalLine belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: label is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means terminalLine performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without terminalLine, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function cachedBuildLine(type, outputPath)

Purpose and intuition: cachedBuildLine belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: type is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. outputPath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means cachedBuildLine performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `cachedBuildLine`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function `fetchLimitedText(url, maxLength = 12000)`

Purpose and intuition: `fetchLimitedText` retrieves `LimitedText` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Parameters and data it receives: `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `maxLength` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and calls a network resource. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with `githubHeaders`, `clampText`, `stripHtmlToText`. Those calls show that `fetchLimitedText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `fetchLimitedText`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

response: `response` stores data returned from a network call. It is initialized from `await fetch(url, {`. Within the function it accesses properties/methods `headers`, `ok`, `text`; passed into `clampText`; assigned or updated later in the function.

rawText: `rawText` stores the completed result of an awaited operation. It is initialized from `clampText(await response.text(), maxLength)`. Within the function it is passed into `stripHtmlToText`; assigned or updated later in the function.

contentType: `contentType` stores a computed value for temporary calculation. It is initialized from `response.headers.get("Content-Type") || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

async function `runOptionalCommand(command, args = [], options = {})`

Purpose and intuition: `runOptionalCommand` executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

Parameters and data it receives: `command` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before

returning or rendering anything. `args` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: No other named helper from this file is detected inside the body. That means `runOptionalCommand` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `runOptionalCommand`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

result: `result` stores the completed result of an awaited operation. It is initialized from `await execFileAsync(command, args, {`. Within the function it is accesses properties/methods `stderr`, `stdout`; assigned or updated later in the function.

function powershellSingleQuoted(value = "")

Purpose and intuition: `powershellSingleQuoted` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means `powershellSingleQuoted` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `powershellSingleQuoted`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

Compiler and IDE workspace

The functions in this group all support compiler and ide workspace. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function sourceLooksCpp(source = "")

Purpose and intuition: sourceLooksCpp belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: source is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means sourceLooksCpp performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without sourceLooksCpp, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

text: text stores a computed value for temporary calculation. It is initialized from String(source || ""). Within the function it is passed into test; assigned or updated later in the function.

function sourceLooksC(source = "")

Purpose and intuition: sourceLooksC belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: source is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means sourceLooksC performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without sourceLooksC, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

text: text stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into test; assigned or updated later in the function.

function detectCodeLanguageFromSource(code = "", fileName = "")

Purpose and intuition: detectCodeLanguageFromSource makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Parameters and data it receives: code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. fileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with languageFromFileName, sourceLooksCpp, sourceLooksC. Those calls show that detectCodeLanguageFromSource is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without detectCodeLanguageFromSource, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

byName: byName stores a computed value for temporary calculation. It is initialized from `languageFromFileName(fileName, code)`. Within the function it is returned to the caller; assigned or updated later in the function.

source: source stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is passed into sourceLooksC, sourceLooksCpp, test; assigned or updated later in the function.

async function findTool(toolName)

Purpose and intuition: findTool belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: toolName is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: Inside the body, it waits for asynchronous work and runs an external command. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

What it calls or works with: It works with `pathExists`, `findExecutableUnder`. Those calls show that `findTool` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `findTool`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

candidates: `candidates` stores a function or callback value for temporary calculation. It is initialized from `(compileToolCandidates[toolName] || [toolName]).filter(Boolean)`. Within the function it is assigned or updated later in the function.

candidate: `candidate` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is returned to the caller; passed into `set`.

command: `command` stores a computed value for temporary calculation. It is initialized from `process.platform === "win32" ? "where" : "command"`. Within the function it is passed into `execFileAsync`; assigned or updated later in the function.

args: `args` stores a computed value for temporary calculation. It is initialized from `process.platform === "win32" ? [candidate] : ["-v", candidate]`. Within the function it is passed into `execFileAsync`; assigned or updated later in the function.

result: `result` stores the completed result of an awaited operation. It is initialized from `await execFileAsync(command, args, { timeout: 5000, windowsHide: true })`. Within the function it is accesses properties/methods `stdout`; passed into `String`; assigned or updated later in the function.

found: `found` stores a function or callback value for lookup collection. It is initialized from `String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean)`. Within the function it is returned to the caller; passed into `set`; assigned or updated later in the function.

found: `found` stores the completed result of an awaited operation. It is initialized from `await findExecutableUnder`. Within the function it is returned to the caller; passed into `set`; assigned or updated later in the function.

found: `found` stores the completed result of an awaited operation. It is initialized from `await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`${toolName}.exe`], 5)`. Within the function it is returned to the caller; passed into `set`; assigned or updated later in the function.

function processTerminalText(result = {})

Purpose and intuition: `processTerminalText` belongs to `compiler` and `ide workspace`. This function supports the `Compile Code workspace`: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means `processTerminalText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `processTerminalText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

elapsedSeconds: `elapsedSeconds` stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

elapsed: `elapsed` stores a computed value for temporary calculation. It is initialized from `elapsedSeconds ? ` in ${elapsedSeconds}s` : ""`. Within the function it is assigned or updated later in the function.

status: `status` stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

output: `output` stores a list for lookup collection. It is initialized from `[result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n")`. Within the function it is assigned or updated later in the function.

function processTerminalTextWithPaths(result = {}, replacements = [])

Purpose and intuition: `processTerminalTextWithPaths` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `replacements` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `replacePathReferences`, `processTerminalText`. Those calls show that `processTerminalTextWithPaths` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `processTerminalTextWithPaths`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function cFamilyCompileProfile(language = "c", fileName = "main.c")

Purpose and intuition: cFamilyCompileProfile belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. fileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means cFamilyCompileProfile performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without cFamilyCompileProfile, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

isCpp: isCpp stores a computed value for temporary calculation. It is initialized from language === "cpp". Within the function it is assigned or updated later in the function.

ext: ext stores a resolved filesystem or route value. It is initialized from path.extname(String(fileName || "").toLowerCase()). Within the function it is passed into includes; assigned or updated later in the function.

header: header stores a list for temporary calculation. It is initialized from [".h", ".hpp", ".hh", ".hxx"].includes(ext). Within the function it is assigned or updated later in the function.

function cFamilyBinaryName(fileName = "program")

Purpose and intuition: cFamilyBinaryName belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: fileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `safeSegment`. Those calls show that `cFamilyBinaryName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `cFamilyBinaryName`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

parsed: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(String(fileName || "program"))`. Within the function it is accesses properties/methods `name`; passed into `safeSegment`; assigned or updated later in the function.

async function toolVersionLine(toolPath = "")

Purpose and intuition: `toolVersionLine` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `toolPath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and runs an external command. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: delegates command execution to `runProcess` so timeout and terminal capture remain consistent.

What it calls or works with: It works with `runProcess`. Those calls show that `toolVersionLine` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `toolVersionLine`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

result: `result` stores a resolved filesystem or route value. It is initialized from `await runProcess(toolPath, ["--version"], { timeoutMs: 7000 })`. Within the function it is accesses properties/methods `stderr`, `stdout`; assigned or updated later in the function.

version: `version` stores a list for temporary calculation. It is initialized from `[result.stdout, result.stderr]`. Within the function it is returned to the caller; passed into `runProcess`, `set`; assigned or updated later in the function.

function cFamilyRunOutput(result = {})

Purpose and intuition: `cFamilyRunOutput` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: result is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means cFamilyRunOutput performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without cFamilyRunOutput, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

elapsedSeconds: elapsedSeconds stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

status: status stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

output: output stores a list for lookup collection. It is initialized from `[result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n")`. Within the function it is assigned or updated later in the function.

function cleanHdlSimulationOutput(result = {})

Purpose and intuition: cleanHdlSimulationOutput belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: result is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means cleanHdlSimulationOutput performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without cleanHdlSimulationOutput, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

raw: raw stores a list for lookup collection. It is initialized from `[result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n")`. Within the function it is accesses properties/methods split; assigned or updated later in the function.

lines: lines stores a function or callback value for lookup collection. It is initialized from `raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean)`. Within the function it is assigned or updated later in the function.

finishLines: finishLines stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods join, length, push; assigned or updated later in the function.

signalLines: signalLines stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods join, length, push; assigned or updated later in the function.

line: line starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods replace, trimEnd; passed into map, push.

elapsedSeconds: elapsedSeconds stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

status: status stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

body: body stores a computed value for temporary calculation. It is initialized from `signalLines.length`. Within the function it is assigned or updated later in the function.

suffix: suffix stores a resolved filesystem or route value. It is initialized from `signalLines.length && finishLines.length ? `\n${finishLines.join("\n")}` : ""`. Within the function it is assigned or updated later in the function.

async function runProcess(command, args = [], options = {})

Purpose and intuition: runProcess is the safe wrapper around external commands. Compilers, Git, Winget, Java, vvp, and installer helpers all eventually need child processes. This function starts the command with controlled options, captures stdout and stderr, enforces a timeout, kills process trees on Windows when necessary, and resolves to a structured result instead of letting raw process behavior leak through the API.

Purpose and intuition: Without this wrapper, every compiler and Git function would need to repeat timeout logic, stdout/stderr collection, and error handling. More importantly, failures would be inconsistent and the frontend terminal would receive unreliable output.

Parameters and data it receives: command is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. args is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it runs an external command and uses a timer to avoid blocking or to wait for another process. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: No other named helper from this file is detected inside the body. That means runProcess performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without runProcess, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

timeoutMs: timeoutMs stores a computed value for temporary calculation. It is initialized from options.timeoutMs || 20000. Within the function it is assigned or updated later in the function.

cwd: cwd stores a computed value for temporary calculation. It is initialized from options.cwd || compileRoot. Within the function it is passed into spawn; assigned or updated later in the function.

startedAt: startedAt stores a computed value for temporary calculation. It is initialized from Date.now(). Within the function it is assigned or updated later in the function.

child: child stores a computed value for temporary calculation. It is initialized from spawn(command, args, {). Within the function it is accesses properties/methods kill, on, pid, stderr, stdin, stdout; passed into execFile; assigned or updated later in the function.

stdout: stdout stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into resolve; assigned or updated later in the function.

stderr: stderr stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into resolve; assigned or updated later in the function.

timedOut: timedOut stores a boolean value for temporary calculation. It is initialized from false. Within the function it is passed into resolve; assigned or updated later in the function.

timer: timer stores a function or callback value for temporary calculation. It is initialized from setTimeout(() => {). Within the function it is passed into clearTimeout; assigned or updated later in the function.

async function compileToolStatus()

Purpose and intuition: compileToolStatus belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `findTool`. Those calls show that `compileToolStatus` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `compileToolStatus`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

tools: `tools` stores a structured object for compiler or language configuration. It is initialized from `{}`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

uniqueTools: `uniqueTools` stores a list for compiler or language configuration. It is initialized from `[...new Set(Object.values(compileLanguageProfiles).flatMap((profile) => profile.primaryTools))]`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

tool: `tool` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into every, `findTool`, `fromEntries`.

languages: `languages` stores a structured object for compiler or language configuration. It is initialized from `{}`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

destructured [id, profile]: `destructured [id, profile]` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent.

required: `required` stores a computed value for temporary calculation. It is initialized from `profile.primaryTools`. Within the function it is iterated or transformed as a collection; accesses properties/methods every, `map`; passed into `fromEntries`; assigned or updated later in the function.

function isHdlLanguage(language = "")

Purpose and intuition: `isHdlLanguage` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Parameters and data it receives: `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `normalizeCodeLanguage`. Those calls show that `isHdlLanguage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `isHdlLanguage`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function inferCompileFileRole(fileName = "", code = "", language = "")

Purpose and intuition: `inferCompileFileRole` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Parameters and data it receives: `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `isHdlLanguage`. Those calls show that `inferCompileFileRole` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `inferCompileFileRole`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

haystack: `haystack` stores a string value for runtime state or cache. It is initialized from ``${fileName}\n${code}`.toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

function normalizeCompileFileRole(value = "", language = "")

Purpose and intuition: `normalizeCompileFileRole` standardizes `CompileFileRole` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `isHdlLanguage`. Those calls show that `normalizeCompileFileRole` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `normalizeCompileFileRole`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[ _s-]+/g, "")`. Within the function it is passed into `includes`; assigned or updated later in the function.

async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code, stdin = "" })

Purpose and intuition: `saveCompileSource` belongs to `compiler` and `ide workspace`. This function supports the `Compile Code workspace`: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `fileId` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. `role` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. `stdin` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: serializes structured data before sending or saving it; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

What it calls or works with: It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeSegment`, `safeCodeFileName`, `resolveInsideCompileRoot`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `saveCompileSource` is not isolated; it is one piece of a larger

workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `saveCompileSource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

lang: `lang` stores a computed value for temporary calculation. It is initialized from `normalizeCodeLanguage(language || detectCodeLanguageFromSource(code, fileName))`. Within the function it is passed into `normalizeCompileFileRole`, `safeCodeFileName`; assigned or updated later in the function.

projectFolder: `projectFolder` stores a computed value for filesystem location. It is initialized from `safeSegment(projectId, "project")`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `resolveInsideCompileRoot`, `writeFile`; assigned or updated later in the function.

codeFileName: `codeFileName` stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(fileName, lang)`. Within the function it is passed into `clampText`, `normalizeCompileFileRole`, `parse`, `resolveInsideCompileRoot`, `safeSegment`; assigned or updated later in the function.

fileFolder: `fileFolder` stores a resolved filesystem or route value. It is initialized from `safeSegment(fileId || path.parse(codeFileName).name, path.parse(codeFileName).name)`. Within the function it is passed into `resolveInsideCompileRoot`, `writeFile`; assigned or updated later in the function.

folder: `folder` stores a computed value for filesystem location. It is initialized from `resolveInsideCompileRoot(projectFolder, fileFolder)`. Within the function it is passed into `mkdir`; assigned or updated later in the function.

sourcePath: `sourcePath` stores a computed value for filesystem location. It is initialized from `resolveInsideCompileRoot(projectFolder, fileFolder, codeFileName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `writeFile`; assigned or updated later in the function.

metadata: `metadata` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; passed into `stringify`; assigned or updated later in the function.

function `javaMainClassName(code = "", fileName = "Main.java")`

Purpose and intuition: `javaMainClassName` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `javaMainClassName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `javaMainClassName`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

source: `source` stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is accesses properties/methods `match`; assigned or updated later in the function.

publicMatch: `publicMatch` stores a computed value for temporary calculation. It is initialized from `source.match(/bpublic\s+class\s+([A-Za-z_$][\w$]*)/)`. Within the function it is returned to the caller; assigned or updated later in the function.

classMatch: `classMatch` stores a computed value for appearance configuration. It is initialized from `source.match(/bclass\s+([A-Za-z_$][\w$]*)/)`. Within the function it is returned to the caller; assigned or updated later in the function.

function compileCacheKey(parts = {})

Purpose and intuition: `compileCacheKey` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `parts` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: serializes structured data before sending or saving it.

What it calls or works with: No other named helper from this file is detected inside the body. That means `compileCacheKey` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `compileCacheKey`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function compileCacheDirectory(projectId, language, key)

Purpose and intuition: `compileCacheDirectory` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing

terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. `key` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `resolveInsideCompileRoot`, `safeSegment`. Those calls show that `compileCacheDirectory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `compileCacheDirectory`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function hdlModuleNames(`code = ""`)

Purpose and intuition: `hdlModuleNames` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means `hdlModuleNames` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `hdlModuleNames`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function hdlHasWaveDump(`code = ""`)

Purpose and intuition: `hdlHasWaveDump` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `hdlHasWaveDump` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `hdlHasWaveDump`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

source: source stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is passed into `test`; assigned or updated later in the function.

function `hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog")`

Purpose and intuition: `hdlFilesFromPayload` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeFileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeLanguage` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `isHdlLanguage`, `safeCodeFileName`, `safeSegment`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `hdlFilesFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `hdlFilesFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

incoming: incoming stores a computed value for temporary calculation. It is initialized from `Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

byId: byId stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `set`, `values`; assigned or updated later in the function.

addFile: addFile stores a function or callback value for temporary calculation. It is initialized from `(item = {}, fallbackId = "") => {}`. Within the function it is assigned or updated later in the function.

code: code stores a computed value for temporary calculation. It is initialized from `String(item.code || "")`. Within the function it is accesses properties/methods `trim`; passed into `String`, `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`; assigned or updated later in the function.

language: language stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`; assigned or updated later in the function.

fileName: fileName stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(item.fileName || activeFileName || compileLanguageProfiles[language].defaultFile, language)`. Within the function it is accesses properties/methods `localeCompare`; passed into `addFile`, `localeCompare`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`, `safeSegment`, `set`; assigned or updated later in the function.

id: id stores a resolved filesystem or route value. It is initialized from `safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name)`. Within the function it is passed into `addFile`, `safeSegment`, `set`; assigned or updated later in the function.

function compileActionFromPayload(value = "", language = "")

Purpose and intuition: `compileActionFromPayload` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `isHdlLanguage`. Those calls show that `compileActionFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `compileActionFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[ _s-]+/g, "-")`. Within the function it is returned to the caller; assigned or updated later in the function.

function compileActionLabel(action = "run", language = "")

Purpose and intuition: `compileActionLabel` belongs to `compiler` and `ide workspace`. This function supports the `Compile Code workspace`: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `action` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `isHdlLanguage`. Those calls show that `compileActionLabel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `compileActionLabel`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "javascript")

Purpose and intuition: `compileWorkspaceFilesFromPayload` belongs to `compiler` and `ide workspace`. This function supports the `Compile Code workspace`: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeFileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeLanguage` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or

output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeCodeFileName`, `safeSegment`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `compileWorkspaceFilesFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `compileWorkspaceFilesFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

incoming: `incoming` stores a computed value for temporary calculation. It is initialized from `Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

byId: `byId` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `set`, `values`; assigned or updated later in the function.

addFile: `addFile` stores a function or callback value for temporary calculation. It is initialized from `(item = {}, fallbackId = "") => {}`. Within the function it is assigned or updated later in the function.

code: `code` stores a computed value for temporary calculation. It is initialized from `String(item.code || "")`. Within the function it accesses properties/methods `trim`; passed into `String`, `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`; assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`; assigned or updated later in the function.

fileName: `fileName` stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(item.fileName || activeFileName || compileLanguageProfiles[language]?.defaultFile || "source.txt", language)`. Within the function it accesses properties/methods `localeCompare`; passed into `addFile`, `localeCompare`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`, `safeSegment`, `set`; assigned or updated later in the function.

id: `id` stores a resolved filesystem or route value. It is initialized from `safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name)`. Within the function it is passed into `addFile`, `safeSegment`, `set`; assigned or updated later in the function.

async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {})

Purpose and intuition: `writeCompileWorkspaceSources` writes `CompileWorkspaceSources` to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

Parameters and data it receives: files points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. targetDir is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

What it calls or works with: It works with normalizeCodeLanguage. Those calls show that writeCompileWorkspaceSources is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without writeCompileWorkspaceSources, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

seen: seen stores a constructed object for lookup collection. It is initialized from new Map(). Within the function it is mutated as a collection or DOM container; accesses properties/methods get, set; assigned or updated later in the function.

written: written stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

file: file starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods code, fileName, language; passed into get, parse, push, set, writeFile.

parsed: parsed stores a resolved filesystem or route value. It is initialized from path.parse(file.fileName). Within the function it is accesses properties/methods ext, name; assigned or updated later in the function.

count: count stores a computed value for temporary calculation. It is initialized from seen.get(file.fileName) || 0. Within the function it is passed into set; assigned or updated later in the function.

uniqueName: uniqueName stores a computed value for temporary calculation. It is initialized from count ? `\${parsed.name}\_\${count}\${parsed.ext}` : file.fileName. Within the function it is passed into join, push; assigned or updated later in the function.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from path.join(targetDir, uniqueName). It controls a boundary or destination for file work, so an incorrect value can redirect uploads,

previews, compiler artifacts, or published assets. Within the function it is passed into push, writeFile; assigned or updated later in the function.

function sourceDisplayName(file = {})

Purpose and intuition: sourceDisplayName belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: file points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means sourceDisplayName performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without sourceDisplayName, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "")

Purpose and intuition: cFamilyWorkspaceSources belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: files points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. activeFileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means cFamilyWorkspaceSources performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `cFamilyWorkspaceSources`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

exts: `exts` stores a computed value for temporary calculation. It is initialized from `language === "cpp"`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

selected: `selected` stores a function or callback value for temporary calculation. It is initialized from `files.filter((file) => exts.has(path.extname(file.fileName).toLowerCase()))`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`, `some`; assigned or updated later in the function.

async function `writeHdlSimulationSources(files = [], cacheDir = "")`

Purpose and intuition: `writeHdlSimulationSources` writes `HdlSimulationSources` to a controlled destination.

Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

Parameters and data it receives: `files` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `cacheDir` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

What it calls or works with: No other named helper from this file is detected inside the body. That means `writeHdlSimulationSources` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `writeHdlSimulationSources`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

sourceDir: `sourceDir` stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "sources")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `mkdir`, `rm`; assigned or updated later in the function.

seen: seen stores a constructed object for lookup collection. It is initialized from new Map(). Within the function it is mutated as a collection or DOM container; accesses properties/methods get, set; assigned or updated later in the function.

written: written stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

file: file starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods code, fileName; passed into get, parse, push, set, writeFile.

parsed: parsed stores a resolved filesystem or route value. It is initialized from path.parse(file.fileName). Within the function it is accesses properties/methods ext, name; assigned or updated later in the function.

count: count stores a computed value for temporary calculation. It is initialized from seen.get(file.fileName) || 0. Within the function it is passed into set; assigned or updated later in the function.

uniqueName: uniqueName stores a computed value for temporary calculation. It is initialized from count ? `\${parsed.name}\_\${count}\${parsed.ext}` : file.fileName. Within the function it is passed into join, push; assigned or updated later in the function.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from path.join(sourceDir, uniqueName). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into push, writeFile; assigned or updated later in the function.

function normalizeVcdValue(raw = "")

Purpose and intuition: normalizeVcdValue standardizes VcdValue data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: raw is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeVcdValue performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeVcdValue, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from String(raw || "").trim(). Within the function it is returned to the caller; accesses properties/methods slice, toLowerCase; assigned or updated later in the function.

function parseVcdScopeText(text = "", source = "waveform.vcd")

Purpose and intuition: parseVcdScopeText reads VCD waveform text from HDL simulation and converts it into signals, time points, values, and scope metadata that a frontend scope viewer can draw. VCD files are compact event logs, not friendly UI data. This parser turns symbol changes over time into understandable signal histories.

Purpose and intuition: The function is essential for the SystemVerilog/Verilog simulator feature. A simulation that produces only text is useful, but a scope view requires structured waveform data. Without this parser, the scope icon would have no signal timeline to show.

Parameters and data it receives: text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. source is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with normalizeVcdValue. Those calls show that parseVcdScopeText is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without parseVcdScopeText, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

lines: lines stores a computed value for temporary calculation. It is initialized from `String(text || "").split(/r?\n/)`. Within the function it is assigned or updated later in the function.

scopes: scopes stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `set`; assigned or updated later in the function.

signalsByCode: signalsByCode stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `get`, `has`, `set`, `size`, `values`; assigned or updated later in the function.

timeScaleParts: timeScaleParts stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

inTimescale: inTimescale stores a boolean value for temporary calculation. It is initialized from `false`. Within the function it is assigned or updated later in the function.

time: time stores a numeric value for temporary calculation. It is initialized from `0`. Within the function it is passed into `max`, `push`; assigned or updated later in the function.

maxTime: maxTime stores a numeric value for temporary calculation. It is initialized from `0`. Within the function it is passed into `max`; assigned or updated later in the function.

definitionDone: definitionDone stores a boolean value for temporary calculation. It is initialized from false. Within the function it is assigned or updated later in the function.

rawLine: rawLine starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods trim.

line: line stores a computed value for temporary calculation. It is initialized from rawLine.trim(). Within the function it is accesses properties/methods includes, match, replace, slice, startsWith; passed into Number, normalizeVcdValue, push; assigned or updated later in the function.

inline: inline stores a computed value for temporary calculation. It is initialized from line.replace("\$timescale", "").replace("\$end", "").trim(). Within the function it is passed into push; assigned or updated later in the function.

scopeMatch: scopeMatch stores a computed value for temporary calculation. It is initialized from line.match(/^\sscope\s+\S+\s+(\.+?)\s+\\$end\$/). Within the function it is passed into push; assigned or updated later in the function.

varMatch: varMatch stores a computed value for temporary calculation. It is initialized from line.match(/^\svar\s+\S+\s+(\d+)\s+(\S+)\s+(\.+?)\s+\\$end\$/). Within the function it is assigned or updated later in the function.

destructured [, width, code, reference]: destructured [, width, code, reference] stores a computed value for imported or unpacked API handles. It is initialized from varMatch. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

cleanReference: cleanReference stores a computed value for temporary calculation. It is initialized from reference.replace(/s+\[([^\]]+)\]/, ""). Within the function it is passed into set; assigned or updated later in the function.

code: code stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into get, set; assigned or updated later in the function.

value: value stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into push; assigned or updated later in the function.

vectorMatch: vectorMatch stores a computed value for temporary calculation. It is initialized from line.match(/^(
[01xz_]+)\s+(\S+)\\$/i). Within the function it is passed into normalizeVcdValue; assigned or updated later in the function.

signal: signal stores a computed value for temporary calculation. It is initialized from signalsByCode.get(code). Within the function it is accesses properties/methods changes; passed into filter; assigned or updated later in the function.

previous: previous stores a computed value for temporary calculation. It is initialized from signal.changes[signal.changes.length - 1]. Within the function it is accesses properties/methods time, value; assigned or updated later in the function.

signals: signals stores a list for temporary calculation. It is initialized from [...signalsByCode.values()]. Within the function it is assigned or updated later in the function.

async function readHdlWaveform(cacheDir = "")

Purpose and intuition: readHdlWaveform reads HdlWaveform from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Parameters and data it receives: cacheDir is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: reads data from disk as part of the operation.

What it calls or works with: It works with findFilesByExtension, parseVcdScopeText. Those calls show that readHdlWaveform is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without readHdlWaveform, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

vcdFiles: vcdFiles stores the completed result of an awaited operation. It is initialized from await findFilesByExtension(cacheDir, ".vcd", 4). Within the function it is iterated or transformed as a collection; accesses properties/methods length, sort; assigned or updated later in the function.

selected: selected stores a function or callback value for temporary calculation. It is initialized from vcdFiles.sort((a, b) => a.length - b.length)[0]. Within the function it is passed into parseVcdScopeText, readFile; assigned or updated later in the function.

text: text stores the completed result of an awaited operation. It is initialized from await readFile(selected, "utf8"). Within the function it is accesses properties/methods length, slice; assigned or updated later in the function.

limited: limited stores a computed value for temporary calculation. It is initialized from text.length > 6_000_000 ? text.slice(0, 6_000_000) : text. Within the function it is passed into parseVcdScopeText; assigned or updated later in the function.

async function clearHdlWaveforms(cacheDir = "")

Purpose and intuition: clearHdlWaveforms belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: cacheDir is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `findFilesByExtension`. Those calls show that `clearHdlWaveforms` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `clearHdlWaveforms`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

vcdFiles: `vcdFiles` stores the completed result of an awaited operation. It is initialized from `await findFilesByExtension(cacheDir, ".vcd", 4)`. Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into `all`; assigned or updated later in the function.

async function compileAndRunCode(payload = {})

Purpose and intuition: This is the central Compile Code brain. It receives the project id, active source file, language, action requested by the user, optional stdin, and any workspace files that belong to the project. From that one payload it decides whether the user is only beautifying code, saving code, compiling, building, running, simulating HDL, or reading waveform output. It is `async` because every serious action here touches the filesystem, discovers tools, writes temporary workspaces, runs external compiler processes, and waits for terminal output.

Purpose and intuition: The function is needed because the builder treats each project like a small IDE workspace. JavaScript, Python, Java, C, C++, Verilog, SystemVerilog, LTspice, and HTML all have different compile/run rules. This function keeps those rules behind one API endpoint so the frontend can ask for a compile operation without knowing every compiler command. Without it, Compile Code would degrade into a text box with no reliable build, run, simulate, cache, or output behavior.

Purpose and intuition: Important internal helpers include `ensureRunSource`, `append`, and `missing`. `ensureRunSource` writes the active source into the compile workspace before a run. `append` converts command results into terminal-style output. `missing` returns a helpful response when a required tool is unavailable. Those small local functions matter because they keep the larger language branches readable.

Parameters and data it receives: `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work, runs an external command and reads or writes files. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must `await` it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; delegates command execution to `runProcess` so timeout and terminal capture remain consistent; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

What it calls or works with: It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeCodeFileName`, `saveCompileSource`, `safeSegment`, `resolveInsideCompileRoot`, `compileActionFromPayload`, `compileWorkspaceFilesFromPayload`. Those calls show that `compileAndRunCode` is not isolated; it is one piece

of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `compileAndRunCode`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

requestedLanguage: `requestedLanguage` stores a computed value for compiler or language configuration. It is initialized from `String(payload.language || "").trim().toLowerCase()`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `normalizeCodeLanguage`; assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `requestedLanguage && requestedLanguage !== "auto"`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `String`, `cFamilyCompileProfile`, `cFamilyWorkspaceSources`, `compileActionFromPayload`, `compileCacheDirectory`, `compileCacheKey`, `compileWorkspaceFilesFromPayload`, `hdlFilesFromPayload`; assigned or updated later in the function.

profile: `profile` stores a computed value for portfolio data state. It is initialized from `compileLanguageProfiles[language] || compileLanguageProfiles.javascript`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `label`; passed into `push`; assigned or updated later in the function.

fileName: `fileName` stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(payload.fileName, language)`. Within the function it is passed into `cFamilyBinaryName`, `cFamilyCompileProfile`, `cFamilyWorkspaceSources`, `compileCacheKey`, `compileWorkspaceFilesFromPayload`, `detectCodeLanguageFromSource`, `hdlFilesFromPayload`; assigned or updated later in the function.

saved: `saved` stores the completed result of an awaited operation. It is initialized from `await saveCompileSource({ ...payload, language, fileName })`. Within the function it is accesses properties/methods `fileName`, `id`, `role`, `sourcePath`, `title`; passed into `cFamilyBinaryName`, `cFamilyCompileProfile`, `cFamilyWorkspaceSources`, `compileCacheKey`, `compileWorkspaceFilesFromPayload`, `hdlFilesFromPayload`, `javaMainClassName`; assigned or updated later in the function.

projectFolder: `projectFolder` stores a computed value for filesystem location. It is initialized from `safeSegment(payload.projectId, "project")`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `resolveInsideCompileRoot`; assigned or updated later in the function.

sourcePath: `sourcePath` stores a computed value for filesystem location. It is initialized from `resolveInsideCompileRoot(projectFolder, safeSegment(saved.id), saved.fileName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `cp`, `push`, `runProcess`; assigned or updated later in the function.

sourceCode: `sourceCode` stores a computed value for temporary calculation. It is initialized from `String(payload.code || "")`. Within the function it is passed into `push`, `writeFile`; assigned or updated later in the function.

action: action stores a computed value for temporary calculation. It is initialized from `compileActionFromPayload(payload.action, language)`. Within the function it is passed into `compileActionFromPayload`, `compileCacheKey`, `push`; assigned or updated later in the function.

allWorkspaceFiles: allWorkspaceFiles stores a computed value for temporary calculation. It is initialized from `compileWorkspaceFilesFromPayload(payload, saved.fileName, language)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`; passed into `cFamilyWorkspaceSources`, `ensureRunSource`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

runDir: runDir stores a string value for filesystem location. It is initialized from `""`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `mkdir`, `runProcess`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

runSourcePath: runSourcePath stores a string value for filesystem location. It is initialized from `""`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `replace`; passed into `cp`, `push`, `runProcess`; assigned or updated later in the function.

runWorkspaceSources: runWorkspaceSources stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `find`, `push`; assigned or updated later in the function.

ensureRunSource: ensureRunSource stores a function or callback value for temporary calculation. It is initialized from `async (options = {}) => {}`. Within the function it is assigned or updated later in the function.

runId: runId stores a string value for temporary calculation. It is initialized from ``${Date.now()}-${Math.random().toString(16).slice(2)}``. Within the function it is passed into `resolveInsideCompileRoot`; assigned or updated later in the function.

filesToWrite: filesToWrite stores a computed value for temporary calculation. It is initialized from `Array.isArray(options.files) && options.files.length ? options.files : allWorkspaceFiles`. Within the function it is passed into `writeCompileWorkspaceSources`; assigned or updated later in the function.

active: active stores a function or callback value for temporary calculation. It is initialized from `runWorkspaceSources.find((file) => file.id === saved.id || file.fileName === saved.fileName)`. Within the function it is assigned or updated later in the function.

stdin: stdin stores a computed value for temporary calculation. It is initialized from `String(payload.stdin || "")`. Within the function it is passed into `String`; assigned or updated later in the function.

forceRebuild: forceRebuild stores a computed value for temporary calculation. It is initialized from `payload.forceRebuild === true`. Within the function it is assigned or updated later in the function.

terminal: terminal stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

append: append stores a function or callback value for temporary calculation. It is initialized from `(label, result) => {}`. Within the function it is assigned or updated later in the function.

missing: missing stores a function or callback value for temporary calculation. It is initialized from `async (tools) => {}`. Within the function it is assigned or updated later in the function.

found: found stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is accesses properties/methods `iverilog`, `java`, `javac`, `vvp`; passed into `append`, `push`, `runProcess`, `toolVersionLine`; assigned or updated later in the function.

tool: tool starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `filter`, `findTool`.

missingTools: missingTools stores a function or callback value for compiler or language configuration. It is initialized from `tools.filter((tool) => !found[tool])`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods `join`, `length`; assigned or updated later in the function.

openTags: openTags stores a function or callback value for temporary calculation. It is initialized from `(String(payload.code || "").match(/<([a-z][\w:-]*)b[>]*>/gi) || []).length`. Within the function it is assigned or updated later in the function.

closeTags: closeTags stores a function or callback value for temporary calculation. It is initialized from `(String(payload.code || "").match(/<\V([a-z][\w:-]*)>/gi) || []).length`. Within the function it is assigned or updated later in the function.

ok: ok stores a computed value for temporary calculation. It is initialized from `openTags >= closeTags`. Within the function it is accesses properties/methods `txt`; passed into `join`, `push`; assigned or updated later in the function.

stdinOptions: stdinOptions stores a computed value for temporary calculation. It is initialized from `stdin ? { input: stdin } : {}`. Within the function it is passed into `runProcess`; assigned or updated later in the function.

ok: ok stores a boolean value for temporary calculation. It is initialized from `false`. Within the function it is accesses properties/methods `txt`; passed into `join`, `push`; assigned or updated later in the function.

node: node stores the completed result of an awaited operation. It is initialized from `await findTool("node")`. Within the function it is passed into `append`, `findTool`, `push`, `runProcess`; assigned or updated later in the function.

jsFiles: jsFiles stores a function or callback value for temporary calculation. It is initialized from `allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "javascript")`. Within the function it is accesses properties/methods `length`; passed into `ensureRunSource`; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from `await ensureRunSource({ files: jsFiles.length ? jsFiles : allWorkspaceFiles, languages: ["javascript"] })`. Within the function it is accesses properties/methods `runDir`, `runSourcePath`, `writtenSources`; passed into `runProcess`; assigned or updated later in the function.

targets: targets stores a computed value for temporary calculation. It is initialized from `action === "build"`. Within the function it is accesses properties/methods `length`; assigned or updated later in the function.

target: target starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `sourcePath`; passed into `runProcess`, `sourceDisplayName`.

check: check stores a resolved filesystem or route value. It is initialized from `await runProcess(node, ["--check", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 })`. Within the function it is accesses

properties/methods ok; passed into processTerminalText, push, runProcess, writeFile; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from await runProcess(node, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions }). Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

python: python stores the completed result of an awaited operation. It is initialized from await findTool("python"). Within the function it is passed into append, ensureRunSource, findTool, push, runProcess; assigned or updated later in the function.

pyFiles: pyFiles stores a function or callback value for temporary calculation. It is initialized from allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "python"). Within the function it is accesses properties/methods length; passed into ensureRunSource; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from await ensureRunSource({ files: pyFiles.length ? pyFiles : allWorkspaceFiles, languages: ["python"] }). Within the function it is accesses properties/methods runDir, runSourcePath, writtenSources; passed into runProcess; assigned or updated later in the function.

targets: targets stores a computed value for temporary calculation. It is initialized from action === "build". Within the function it is accesses properties/methods length; assigned or updated later in the function.

target: target starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods sourcePath; passed into runProcess, sourceDisplayName.

check: check stores a resolved filesystem or route value. It is initialized from await runProcess(python, ["-m", "py_compile", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 }). Within the function it is accesses properties/methods ok; passed into processTerminalText, push, runProcess, writeFile; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from await runProcess(python, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions }). Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

cProfile: cProfile stores a computed value for portfolio data state. It is initialized from cFamilyCompileProfile(language, saved.fileName). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods compiler, flags, header, label, languageFlag, standard; passed into push, writeFile; assigned or updated later in the function.

toolName: toolName stores a computed value for compiler or language configuration. It is initialized from cProfile.compiler. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into missing, push, runProcess, toolVersionLine; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from await missing([toolName]). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

workspaceSources: workspaceSources stores a computed value for temporary calculation. It is initialized from cFamilyWorkspaceSources(allWorkspaceFiles, language, saved.fileName). Within the function it is iterated or transformed as a collection; accesses properties/methods filter; passed into writeCompileWorkspaceSources; assigned or updated later in the function.

buildTargets: buildTargets stores a computed value for temporary calculation. It is initialized from action === "build". Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into compileCacheKey; assigned or updated later in the function.

compilerVersion: compilerVersion stores the completed result of an awaited operation. It is initialized from await toolVersionLine(found[toolName]). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into push; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from compileCacheKey({}). Within the function it is passed into compileCacheDirectory; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from compileCacheDirectory(payload.projectId, language, cacheKey). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into clearHdlWaveforms, join, mkdir, readHdlWaveform, runProcess, writeHdlSimulationSources; assigned or updated later in the function.

workspaceDir: workspaceDir stores a resolved filesystem or route value. It is initialized from path.join(cacheDir, "sources"). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into rm, runProcess, writeCompileWorkspaceSources; assigned or updated later in the function.

writtenSources: writtenSources stores the completed result of an awaited operation. It is initialized from await writeCompileWorkspaceSources(workspaceSources, workspaceDir). Within the function it is iterated or transformed as a collection; accesses properties/methods filter, find, length, map; passed into processTerminalTextWithPaths, push; assigned or updated later in the function.

activeWritten: activeWritten stores a function or callback value for temporary calculation. It is initialized from writtenSources.find((file) => file.fileName === saved.fileName) || writtenSources[0]. Within the function it is assigned or updated later in the function.

binaryName: binaryName stores a computed value for temporary calculation. It is initialized from cFamilyBinaryName(saved.fileName). Within the function it is passed into join, push; assigned or updated later in the function.

output: output stores a resolved filesystem or route value. It is initialized from path.join(cacheDir, binaryName). Within the function it is passed into pathExists, push, runProcess; assigned or updated later in the function.

outputMarker: outputMarker stores a resolved filesystem or route value. It is initialized from path.join(cacheDir, "syntax-ok.txt"). Within the function it is passed into pathExists, push, writeFile; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from !forceRebuild && await pathExists(output). Within the function it is assigned or updated later in the function.

cachedHeader: cachedHeader stores the completed result of an awaited operation. It is initialized from cProfile.header && !forceRebuild && await pathExists(outputMarker). Within the function it is assigned or updated later in the function.

args: args stores a computed value for temporary calculation. It is initialized from cProfile.header. Within the function it is passed into runProcess; assigned or updated later in the function.

compile: compile stores the completed result of an awaited operation. It is initialized from await runProcess(found[toolName], args, { cwd: workspaceDir, timeoutMs: 30000 }). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods ok; passed into processTerminalTextWithPaths, push; assigned or updated later in the function.

replacements: replacements stores a list for temporary calculation. It is initialized from [. Within the function it is passed into processTerminalTextWithPaths; assigned or updated later in the function.

toolPath: toolPath stores a computed value for filesystem location. It is initialized from found[toolName]. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into runProcess; assigned or updated later in the function.

run: run stores the completed result of an awaited operation. It is initialized from await runProcess(output, [], {. Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from await missing(["javac", "java"]). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

className: className stores a computed value for appearance configuration. It is initialized from javaMainClassName(payload.code, saved.fileName). Within the function it is passed into join, runProcess; assigned or updated later in the function.

javaFiles: javaFiles stores a function or callback value for temporary calculation. It is initialized from allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "java"). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; passed into compileCacheKey, writeCompileWorkspaceSources; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from compileCacheKey({. Within the function it is passed into compileCacheDirectory; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from compileCacheDirectory(payload.projectId, language, cacheKey). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into clearHdlWaveforms, join, mkdir, readHdlWaveform, runProcess, writeHdlSimulationSources; assigned or updated later in the function.

classPath: classPath stores a resolved filesystem or route value. It is initialized from path.join(cacheDir, `\${className}.class`). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into pathExists, push; assigned or updated later in the function.

sourceDir: sourceDir stores a resolved filesystem or route value. It is initialized from path.join(cacheDir, "sources"). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into join, rm, runProcess, writeCompileWorkspaceSources; assigned or updated later in the function.

writtenSources: writtenSources stores the completed result of an awaited operation. It is initialized from `await writeCompileWorkspaceSources(javaFiles.length ? javaFiles : allWorkspaceFiles, sourceDir, { languages: ["java"] })`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `find`, `length`, `map`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

javaPath: javaPath stores a function or callback value for filesystem location. It is initialized from `writtenSources.find((file) => file.fileName === saved.fileName)?.sourcePath || path.join(sourceDir, `${className}.java`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `basename`, `writeFile`; assigned or updated later in the function.

cached: cached stores a resolved filesystem or route value. It is initialized from `!forceRebuild && await pathExists(classPath)`. Within the function it is assigned or updated later in the function.

compileTargets: compileTargets stores a function or callback value for compiler or language configuration. It is initialized from `action === "build" ? writtenSources.map((file) => file.sourcePath) : [javaPath]`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `runProcess`; assigned or updated later in the function.

compile: compile stores the completed result of an awaited operation. It is initialized from `await runProcess(found.javac, ["-d", cacheDir, ...compileTargets], { cwd: sourceDir, timeoutMs: 30000 })`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods `ok`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

run: run stores the completed result of an awaited operation. It is initialized from `await runProcess(found.java, ["-cp", cacheDir, className], { cwd: cacheDir, timeoutMs: 20000, ...stdinOptions })`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from `await missing(["iverilog", "vvp"])`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

hdlFiles: hdlFiles stores a computed value for temporary calculation. It is initialized from `hdlFilesFromPayload(payload, saved.fileName, language)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `map`, `some`; passed into `compileCacheKey`, `writeHdlSimulationSources`; assigned or updated later in the function.

testbenchFiles: testbenchFiles stores a function or callback value for temporary calculation. It is initialized from `hdlFiles.filter((file) => file.role === "testbench")`. Within the function it is accesses properties/methods `flatMap`, `length`, `some`; passed into `push`; assigned or updated later in the function.

usesSystemVerilog: usesSystemVerilog stores a function or callback value for temporary calculation. It is initialized from `hdlFiles.some((file) => normalizeCodeLanguage(file.language) === "systemverilog")`. Within the function it is passed into `compileCacheKey`; assigned or updated later in the function.

flags: flags stores a computed value for temporary calculation. It is initialized from `usesSystemVerilog ? ["-g2012", "-Wall"] : ["-g2005-sv", "-Wall"]`. Within the function it is accesses properties/methods `join`; assigned or updated later in the function.

topModule: topModule stores a function or callback value for lookup collection. It is initialized from `testbenchFiles.flatMap((file) => hdlModuleNames(file.code)).find(Boolean) || ""`. Within the function it is passed into push; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from `compileCacheKey({}`. Within the function it is passed into `compileCacheDirectory`; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from `compileCacheDirectory(payload.projectId, language, cacheKey)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `clearHdlWaveforms`, `join`, `mkdir`, `readHdlWaveform`, `runProcess`, `writeHdlSimulationSources`; assigned or updated later in the function.

output: output stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "simulation.vvp")`. Within the function it is passed into `pathExists`, `push`, `runProcess`; assigned or updated later in the function.

sourceFiles: sourceFiles stores the completed result of an awaited operation. It is initialized from `await writeHdlSimulationSources(hdlFiles, cacheDir)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `map`; assigned or updated later in the function.

designCount: designCount stores a function or callback value for temporary calculation. It is initialized from `sourceFiles.filter((file) => file.role !== "testbench").length`. Within the function it is passed into push; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from `!forceRebuild && await pathExists(output)`. Within the function it is assigned or updated later in the function.

compileArgs: compileArgs stores a list for compiler or language configuration. It is initialized from `[]`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `runProcess`; assigned or updated later in the function.

compile: compile stores the completed result of an awaited operation. It is initialized from `await runProcess(found.iverilog, compileArgs, { cwd: cacheDir, timeoutMs: 30000 })`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods `ok`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

replacements: replacements stores a function or callback value for lookup collection. It is initialized from `sourceFiles.map((file) => ({ from: file.sourcePath, to: file.uniqueName }))`. Within the function it is passed into `processTerminalTextWithPaths`; assigned or updated later in the function.

waveform: waveform stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is accesses properties/methods `maxTime`, `signals`, `source`, `timeScale`, `vcd`; passed into `Boolean`, `dumpfile`, `push`; assigned or updated later in the function.

run: run stores the completed result of an awaited operation. It is initialized from `await runProcess(found.vvp, [output], { cwd: cacheDir, timeoutMs: 20000 })`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

ItsPice: ItsPice stores the completed result of an awaited operation. It is initialized from `await findTool("ItsPice")`. Within the function it is passed into `append`, `findTool`, `runProcess`; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from await ensureRunSource(). Within the function it is accesses properties/methods runDir, runSourcePath, writtenSources; passed into runProcess; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from await runProcess(ltspice, ["-b", runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 45000 }). Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

logPath: logPath stores a resolved filesystem or route value. It is initialized from runSource.runSourcePath.replace(/\.([^.]+\\$/ , ".log"). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into push; assigned or updated later in the function.

async function installCompilerTools(language = "")

Purpose and intuition: installCompilerTools installs or prepares external tooling. It belongs in the backend because installation touches the machine, not just the browser page.

Parameters and data it receives: language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: delegates command execution to runProcess so timeout and terminal capture remain consistent.

What it calls or works with: It works with normalizeCodeLanguage, findTool, runProcess, terminalLine, processTerminalText, compileToolStatus. Those calls show that installCompilerTools is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without installCompilerTools, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

lang: lang stores a computed value for temporary calculation. It is initialized from normalizeCodeLanguage(language). Within the function it is assigned or updated later in the function.

profile: profile stores a computed value for portfolio data state. It is initialized from compileLanguageProfiles[lang] || compileLanguageProfiles.javascript. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods label, winget; assigned or updated later in the function.

winget: winget stores the completed result of an awaited operation. It is initialized from await findTool("winget") || "winget". Within the function it is passed into findTool, push, runProcess; assigned or updated later in the function.

terminal: terminal stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods join, push; assigned or updated later in the function.

ok: ok stores a boolean value for temporary calculation. It is initialized from true. Within the function it is assigned or updated later in the function.

packageld: packageld starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

args: args stores a list for temporary calculation. It is initialized from []. Within the function it is accesses properties/methods join; passed into push, runProcess; assigned or updated later in the function.

result: result stores the completed result of an awaited operation. It is initialized from await runProcess(winget, args, { cwd: root, timeoutMs: 600000 }). Within the function it is accesses properties/methods ok; passed into processTerminalText; assigned or updated later in the function.

function sourceLooksTextual(source = {})

Purpose and intuition: sourceLooksTextual belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: source is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means sourceLooksTextual performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without sourceLooksTextual, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

url: url stores a computed value for network or routing value. It is initialized from String(source.url || ""). Within the function it is passed into String, test; assigned or updated later in the function.

kind: kind stores a computed value for temporary calculation. It is initialized from String(source.kind || "").toLowerCase(). Within the function it is passed into String; assigned or updated later in the function.

function sourceUrlAllowed(url)

Purpose and intuition: sourceUrlAllowed belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means `sourceUrlAllowed` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sourceUrlAllowed`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(url)`. Within the function it is accesses properties/methods `hostname`; assigned or updated later in the function.

hostName: `hostName` stores a computed value for temporary calculation. It is initialized from `parsed.hostname.toLowerCase()`. Within the function it is passed into `includes`; assigned or updated later in the function.

githubTextFilePattern: `githubTextFilePattern` stores a computed value for Git publishing and authorization state. It is initialized from `/(.txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|cc|hh|py|js|mjs|ts|tsx|v|sv|vhdl?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb|ino|xdc|sdc|tcl|yaml|yml)$/i`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

githubSkipFilePattern: `githubSkipFilePattern` stores a computed value for Git publishing and authorization state. It is initialized from `/(.png|jpe?g|gif|webp|svg|pdf|zip|7z|rar|exe|dll|bin|obj|o|a|so|dylib|mp4|mov|avi|mp3|wav|xlsx?|pptx?|docx?)$/i`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

function parseGitHubSourceUrl(url)

Purpose and intuition: `parseGitHubSourceUrl` interprets `GitHubSourceUrl` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means `parseGitHubSourceUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `parseGitHubSourceUrl`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(url)`. Within the function it is accesses properties/methods `hostname`, `pathname`; assigned or updated later in the function.

host: `host` stores a computed value for temporary calculation. It is initialized from `parsed.hostname.toLowerCase()`. Within the function it is assigned or updated later in the function.

parts: `parts` stores a computed value for temporary calculation. It is initialized from `parsed.pathname.split("/").filter(Boolean)`. Within the function it is accesses properties/methods `length`, `slice`; assigned or updated later in the function.

base: `base` stores a structured object for temporary calculation. It is initialized from `{ owner: parts[0], repo: parts[1] }`. Within the function it is assigned or updated later in the function.

async function `fetchGitHubProfileSource(source = {}, parsed = {})`

Purpose and intuition: `fetchGitHubProfileSource` retrieves `GitHubProfileSource` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Parameters and data it receives: `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `parsed` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `fetchGitHubJson`, `wantsGitHubCode`, `scoreGitHubRepo`, `fetchGitHubRepositorySource`, `clampText`. Those calls show that `fetchGitHubProfileSource` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `fetchGitHubProfileSource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

owner: `owner` stores a computed value for temporary calculation. It is initialized from `parsed.owner`. Within the function it is passed into `all`, `fetchGitHubJson`, `fetchGitHubRepositorySource`; assigned or updated later in the function.

destructured [profile, repos]: destructured [profile, repos] stores the completed result of an awaited operation. It is initialized from `await Promise.all([`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent.

repoList: repoList stores a computed value for temporary calculation. It is initialized from `Array.isArray(repos) ? repos : []`. Within the function it is accesses properties/methods `slice`; assigned or updated later in the function.

includeCode: includeCode stores a computed value for temporary calculation. It is initialized from `wantsGitHubCode(source.question || "")`. Within the function it is assigned or updated later in the function.

selectedRepos: selectedRepos stores a computed value for temporary calculation. It is initialized from `includeCode`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

repoBlocks: repoBlocks stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `flatMap`, `push`; assigned or updated later in the function.

repo: repo starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `default_branch`, `description`, `full_name`, `html_url`, `language`, `name`, `updated_at`; passed into `fetchGitHubRepositorySource`, `map`.

repoSource: repoSource stores the completed result of an awaited operation. It is initialized from `await fetchGitHubRepositorySource({`. Within the function it is accesses properties/methods `text`; passed into `push`; assigned or updated later in the function.

lines: lines stores a list for temporary calculation. It is initialized from `[]`. Within the function it is accesses properties/methods `join`; passed into `clampText`; assigned or updated later in the function.

async function fetchGitHubRepositorySource(source = {}, parsed = {})

Purpose and intuition: `fetchGitHubRepositorySource` retrieves `GitHubRepositorySource` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Parameters and data it receives: `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `parsed` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `fetchGitHubJson`, `wantsGitHubCode`, `fetchLimitedText`, `clampText`, `scoreGitHubFile`. Those calls show that `fetchGitHubRepositorySource` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `fetchGitHubRepositorySource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

destructured { owner, repo }: destructured { owner, repo } stores a computed value for imported or unpacked API handles. It is initialized from `parsed`. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

metadata: metadata stores the completed result of an awaited operation. It is initialized from `await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`)`. Within the function it is accesses properties/methods `default_branch`, `description`, `html_url`, `language`; assigned or updated later in the function.

branch: branch stores a computed value for Git publishing and authorization state. It is initialized from `parsed.branch || metadata?.default_branch || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `encodeURIComponent`, `fetchLimitedText`; assigned or updated later in the function.

question: question stores a computed value for temporary calculation. It is initialized from `source.question || ""`. Within the function it is passed into `Number`, `scoreGitHubFile`, `wantsGitHubCode`; assigned or updated later in the function.

requestedMaxCodeFiles: requestedMaxCodeFiles stores a computed value for temporary calculation. It is initialized from `Number(source.maxCodeFiles || (wantsGitHubCode(question) ? 6 : 3))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxCodeFiles: maxCodeFiles stores a computed value for temporary calculation. It is initialized from `Math.max(1, Math.min(8, Number.isFinite(requestedMaxCodeFiles) ? requestedMaxCodeFiles : 3))`. Within the function it is passed into `Number`, `slice`; assigned or updated later in the function.

requestedFileTextLength: requestedFileTextLength stores a computed value for temporary calculation. It is initialized from `Number(source.maxFileTextLength || (wantsGitHubCode(question) ? 7000 : 3000))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxFileTextLength: maxFileTextLength stores a computed value for temporary calculation. It is initialized from `Math.max(1000, Math.min(10000, Number.isFinite(requestedFileTextLength) ? requestedFileTextLength : 3000))`. Within the function it is passed into `Number`, `fetchLimitedText`; assigned or updated later in the function.

requestedRepoTextLength: requestedRepoTextLength stores a computed value for temporary calculation. It is initialized from `Number(source.maxRepositoryTextLength || (wantsGitHubCode(question) ? 22000 : 14000))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxRepositoryTextLength: maxRepositoryTextLength stores a computed value for temporary calculation. It is initialized from `Math.max(5000, Math.min(26000, Number.isFinite(requestedRepoTextLength) ? requestedRepoTextLength : 14000))`. Within the function it is passed into `Number`; assigned or updated later in the function.

lines: lines stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; passed into `clampText`; assigned or updated later in the function.

rawUrl: rawUrl stores a string value for network or routing value. It is initialized from ``https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${parsed.filePath}``. Within the function it is passed into `fetchLimitedText`; assigned or updated later in the function.

text: text stores the completed result of an awaited operation. It is initialized from `await fetchLimitedText(rawUrl, 14000)`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

tree: tree stores the completed result of an awaited operation. It is initialized from `await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/git/trees/${encodeURIComponent(branch)}?recursive=1`)`. Within the function it is accesses properties/methods `tree`; passed into `isArray`; assigned or updated later in the function.

files: files stores a computed value for temporary calculation. It is initialized from `Array.isArray(tree?.tree)`. Within the function it is accesses properties/methods `find`, `length`, `slice`; passed into `push`; assigned or updated later in the function.

readmePath: readmePath stores a function or callback value for filesystem location. It is initialized from `files.find((file) => /^(^|V)readme\.md$/i.test(file.path)).path || "README.md"`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `fetchLimitedText`, `push`; assigned or updated later in the function.

readmeText: readmeText stores a function or callback value for temporary calculation. It is initialized from `await fetchLimitedText(`https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${readmePath}`, 9000).catch(() => "")`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

includeCode: includeCode stores a computed value for temporary calculation. It is initialized from `wantsGitHubCode(question)`. Within the function it is assigned or updated later in the function.

selectedFiles: selectedFiles stores a computed value for temporary calculation. It is initialized from `files`. Within the function it is assigned or updated later in the function.

file: file starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `path`; passed into `filter`, `find`, `map`, `push`, `test`.

rawUrl: rawUrl stores a string value for network or routing value. It is initialized from ``https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${file.path}``. Within the function it is passed into `fetchLimitedText`; assigned or updated later in the function.

fileText: fileText stores a function or callback value for temporary calculation. It is initialized from `await fetchLimitedText(rawUrl, maxFileTextLength).catch(() => "")`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

async function fetchGitHubSourceText(source = {})

Purpose and intuition: `fetchGitHubSourceText` retrieves `GitHubSourceText` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Parameters and data it receives: `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `parseGitHubSourceUrl`, `fetchGitHubProfileSource`, `fetchGitHubRepositorySource`. Those calls show that `fetchGitHubSourceText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `fetchGitHubSourceText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

parsed: `parsed` stores a computed value for temporary calculation. It is initialized from `parseGitHubSourceUrl(source.url || "")`. Within the function it is accessed properties/methods type; passed into `fetchGitHubProfileSource`, `fetchGitHubRepositorySource`; assigned or updated later in the function.

async function readLocalSourceText(url)

Purpose and intuition: `readLocalSourceText` reads `LocalSourceText` from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Parameters and data it receives: `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns the awaited result of another asynchronous operation, so its caller receives the completed value rather than a pending Promise. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: reads data from disk as part of the operation.

What it calls or works with: It works with `resolveInsideRoot`. Those calls show that `readLocalSourceText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `readLocalSourceText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

parsed: `parsed` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accessed properties/methods `hostname`, `pathname`; passed into `decodeURIComponent`; assigned or updated later in the function.

localHosts: localHosts stores a constructed object for lookup collection. It is initialized from new Set(["localhost", "127.0.0.1"]). Within the function it is accesses properties/methods has; assigned or updated later in the function.

pathname: pathname stores a computed value for filesystem location. It is initialized from decodeURIComponent(parsed.pathname.replace(/^\//, "")). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods includes, replace; passed into decodeURIComponent, extname, readFile; assigned or updated later in the function.

ext: ext stores a resolved filesystem or route value. It is initialized from path.extname(pathname).toLowerCase(). Within the function it is assigned or updated later in the function.

async function fetchSourceText(source = {})

Purpose and intuition: fetchSourceText retrieves SourceText from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Parameters and data it receives: source is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with sourceLooksTextual, sourceUrlAllowed, fetchGitHubSourceText, readLocalSourceText, clampText, stripHtmlToText. Those calls show that fetchSourceText is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without fetchSourceText, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

url: url stores a computed value for network or routing value. It is initialized from String(source.url || ""). Within the function it is passed into String, fetch, readLocalSourceText, sourceUrlAllowed; assigned or updated later in the function.

githubText: githubText stores the completed result of an awaited operation. It is initialized from await fetchGitHubSourceText(source). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; assigned or updated later in the function.

localText: localText stores the completed result of an awaited operation. It is initialized from await readLocalSourceText(url). Within the function it is accesses properties/methods replace, trim; passed into clampText; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch(url, {`. Within the function it accesses properties/methods `headers`, `ok`, `text`; passed into `clampText`; assigned or updated later in the function.

contentType: `contentType` stores a computed value for temporary calculation. It is initialized from `response.headers.get("Content-Type") || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

rawText: `rawText` stores the completed result of an awaited operation. It is initialized from `clampText(await response.text(), 12000)`. Within the function it is passed into `stripHtmlToText`; assigned or updated later in the function.

text: `text` stores a computed value for temporary calculation. It is initialized from `/html/i.test(contentType) ? stripHtmlToText(rawText) : rawText`. Within the function it accesses properties/methods `trim`; passed into `clampText`, `fetch`; assigned or updated later in the function.

function resolveInsideCompileRoot(...segments)

Purpose and intuition: `resolveInsideCompileRoot` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: `segments` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: No other named helper from this file is detected inside the body. That means `resolveInsideCompileRoot` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `resolveInsideCompileRoot`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

target: `target` stores a resolved filesystem or route value. It is initialized from `path.resolve(compileRoot, ...segments)`. Within the function it is returned to the caller; passed into `relative`; assigned or updated later in the function.

relative: `relative` stores a resolved filesystem or route value. It is initialized from `path.relative(compileRoot, target)`. Within the function it accesses properties/methods `startsWith`; passed into `isAbsolute`; assigned or updated later in the function.

Filesystem safety

The functions in this group all support filesystem safety. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function languageFromFileName(fileName = "", code = "")

Purpose and intuition: languageFromFileName belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: fileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with sourceLooksCpp, sourceLooksC. Those calls show that languageFromFileName is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without languageFromFileName, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

ext: ext stores a resolved filesystem or route value. It is initialized from path.extname(String(fileName || "").toLowerCase()). Within the function it is passed into includes; assigned or updated later in the function.

function safeCodeFileName(value = "", language = "javascript")

Purpose and intuition: safeCodeFileName is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with normalizeCodeLanguage, safeSegment. Those calls show that safeCodeFileName is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without safeCodeFileName, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

profile: profile stores a computed value for portfolio data state. It is initialized from `compileLanguageProfiles[normalizeCodeLanguage(language)] || compileLanguageProfiles.javascript`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `defaultFile`, `extensions`; assigned or updated later in the function.

fallback: fallback stores a computed value for temporary calculation. It is initialized from `profile.defaultFile`. Within the function it is passed into `String`, `extname`, `parse`, `safeSegment`; assigned or updated later in the function.

parsed: parsed stores a resolved filesystem or route value. It is initialized from `path.parse(String(value || fallback))`. Within the function it is accesses properties/methods `ext`, `name`; passed into `String`, `safeSegment`; assigned or updated later in the function.

safeName: safeName stores a resolved filesystem or route value. It is initialized from `safeSegment(parsed.name, path.parse(fallback).name)`. Within the function it is assigned or updated later in the function.

ext: ext stores a resolved filesystem or route value. It is initialized from `String(parsed.ext || path.extname(fallback)).toLowerCase()`. Within the function it is passed into `String`; assigned or updated later in the function.

function pathVariantsForReplacement(value = "")

Purpose and intuition: `pathVariantsForReplacement` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `pathVariantsForReplacement` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `pathVariantsForReplacement`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(value || "")`. Within the function it is accesses properties/methods `replace`; passed into `from`, `normalize`; assigned or updated later in the function.

function replacePathReferences(text = "", replacements = [])

Purpose and intuition: replacePathReferences belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. replacements is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with pathVariantsForReplacement. Those calls show that replacePathReferences is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without replacePathReferences, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

output: output stores a computed value for temporary calculation. It is initialized from String(text || ""). Within the function it is returned to the caller; accesses properties/methods split; assigned or updated later in the function.

replacement: replacement starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

from: from stores a computed value for temporary calculation. It is initialized from replacement?.from || "". Within the function it is assigned or updated later in the function.

to: to stores a computed value for temporary calculation. It is initialized from replacement?.to || "". Within the function it is passed into join; assigned or updated later in the function.

variant: variant starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into split.

async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3)

Purpose and intuition: findFilesByExtension belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: folder is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. extension is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a

predictable empty result when the caller omits that field. `maxDepth` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

What it calls or works with: It works with `pathExists`. Those calls show that `findFilesByExtension` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `findFilesByExtension`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

entries: `entries` stores a function or callback value for temporary calculation. It is initialized from `await readdir(folder, { withFileTypes: true }).catch(() => [])`. Within the function it is assigned or updated later in the function.

matches: `matches` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

entry: `entry` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accessed properties/methods `isDirectory`, `isFile`, `name`; passed into `join`.

fullPath: `fullPath` stores a resolved filesystem or route value. It is initialized from `path.join(folder, entry.name)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`; assigned or updated later in the function.

async function readJsonFile(filePath)

Purpose and intuition: `readJsonFile` reads `JsonFile` from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Parameters and data it receives: `filePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: parses JSON rather than treating incoming text as already trusted data; reads data from disk as part of the operation.

What it calls or works with: No other named helper from this file is detected inside the body. That means readJsonFile performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without readJsonFile, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function safeSegment(value, fallback = "item")

Purpose and intuition: safeSegment is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. fallback is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means safeSegment performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without safeSegment, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

segment: segment stores a computed value for temporary calculation. It is initialized from String(value || fallback). Within the function it is returned to the caller; assigned or updated later in the function.

function safeFileName(value)

Purpose and intuition: safeFileName is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `safeSegment`. Those calls show that `safeFileName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `safeFileName`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

parsed: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(String(value || "file"))`. Within the function it is accesses properties/methods `ext`, `name`; passed into `safeSegment`; assigned or updated later in the function.

name: `name` stores a computed value for temporary calculation. It is initialized from `safeSegment(parsed.name, "file")`. Within the function it is passed into `safeSegment`; assigned or updated later in the function.

ext: `ext` stores a computed value for temporary calculation. It is initialized from `safeSegment(parsed.ext.replace(".", ""), "")`. Within the function it is returned to the caller; accesses properties/methods `replace`; passed into `safeSegment`; assigned or updated later in the function.

function resolveInsideRoot(...segments)

Purpose and intuition: `resolveInsideRoot` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: `segments` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: No other named helper from this file is detected inside the body. That means `resolveInsideRoot` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `resolveInsideRoot`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

target: `target` stores a resolved filesystem or route value. It is initialized from `path.normalize(path.join(root, ...segments))`. Within the function it is returned to the caller; accesses properties/methods `startsWith`; assigned or updated later in the function.

function resolveInsidePortfolioRoot(...segments)

Purpose and intuition: resolveInsidePortfolioRoot is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: segments is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: No other named helper from this file is detected inside the body. That means resolveInsidePortfolioRoot performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without resolveInsidePortfolioRoot, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

target: target stores a resolved filesystem or route value. It is initialized from path.resolve(portfolioRoot, ...segments). Within the function it is returned to the caller; passed into relative; assigned or updated later in the function.

relative: relative stores a resolved filesystem or route value. It is initialized from path.relative(portfolioRoot, target). Within the function it is accesses properties/methods startsWith; passed into isAbsolute; assigned or updated later in the function.

function samePath(left, right)

Purpose and intuition: samePath belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: left is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. right is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

What it calls or works with: No other named helper from this file is detected inside the body. That means samePath performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `samePath`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function pathExists(filePath)

Purpose and intuition: `pathExists` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `filePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: No other named helper from this file is detected inside the body. That means `pathExists` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `pathExists`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot)

Purpose and intuition: `workspaceHasCompatibleSiteFiles` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `baseRoot` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; reads data from disk as part of the operation.

What it calls or works with: No other named helper from this file is detected inside the body. That means `workspaceHasCompatibleSiteFiles` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `workspaceHasCompatibleSiteFiles`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

fileName: fileName starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into readFile.

Public source and AI context

The functions in this group all support public source and ai context. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function gitHubHeaders(accept = "application/vnd.github+json")

Purpose and intuition: gitHubHeaders belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: accept is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means gitHubHeaders performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without gitHubHeaders, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function fetchGitHubJson(url)

Purpose and intuition: fetchGitHubJson retrieves GitHubJson from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Parameters and data it receives: url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it waits for asynchronous work and calls a network resource. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with gitHubHeaders. Those calls show that fetchGitHubJson is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `fetchGitHubJson`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

response: response stores data returned from a network call. It is initialized from `await fetch(url, { headers: githubHeaders() })`. Within the function it is returned to the caller; accesses properties/methods `json`, `ok`; assigned or updated later in the function.

function githubQuestionTokens(question = "")

Purpose and intuition: `githubQuestionTokens` belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means `githubQuestionTokens` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `githubQuestionTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function scoreGitHubFile(filePath = "", question = "")

Purpose and intuition: `scoreGitHubFile` ranks a candidate against a question or search term. Scoring functions are what make search and AI source selection feel relevant instead of random.

Parameters and data it receives: `filePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `githubQuestionTokens`. Those calls show that `scoreGitHubFile` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `scoreGitHubFile`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

cleanPath: `cleanPath` stores a resolved filesystem or route value. It is initialized from `filePath.toLowerCase()`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accessed; properties/methods included; passed into tests; assigned or updated later in the function.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `githubQuestionTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

score: `score` stores a resolved filesystem or route value. It is initialized from `githubTextFilePattern.test(cleanPath) ? 10 : 0`. Within the function it is returned to the caller; assigned or updated later in the function.

function wantsGitHubCode(question = "")

Purpose and intuition: `wantsGitHubCode` belongs to public source and AI context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `wantsGitHubCode` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `wantsGitHubCode`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function scoreGitHubRepo(repo = {}, question = "")

Purpose and intuition: `scoreGitHubRepo` ranks a candidate against a question or search term. Scoring functions are what make search and AI source selection feel relevant instead of random.

Parameters and data it receives: `repo` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `githubQuestionTokens`. Those calls show that `scoreGitHubRepo` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `scoreGitHubRepo`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `githubQuestionTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

haystack: `haystack` stores a list for runtime state or cache. It is initialized from `[]`. Within the function it is accessed; properties/methods `includes`; passed into `test`; assigned or updated later in the function.

score: `score` stores a computed value for temporary calculation. It is initialized from `repo.fork ? -10 : 12`. Within the function it is returned to the caller; assigned or updated later in the function.

async function enrichPortfolioContext(context = {})

Purpose and intuition: `enrichPortfolioContext` belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: No other named helper from this file is detected inside the body. That means `enrichPortfolioContext` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `enrichPortfolioContext`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

question: `question` stores a computed value for temporary calculation. It is initialized from `String(context.question || "")`. Within the function it is passed into `String`; assigned or updated later in the function.

sources: sources stores a computed value for temporary calculation. It is initialized from `Array.isArray(context.sourceFetches)`. Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into all; assigned or updated later in the function.

sourceExcerpts: sourceExcerpts stores a function or callback value for lookup collection. It is initialized from `(await Promise.all(sources.map(fetchSourceText))).filter(Boolean)`. Within the function it is assigned or updated later in the function.

function parseGitHubRemote(remoteUrl = "")

Purpose and intuition: parseGitHubRemote interprets GitHubRemote text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: remoteUrl points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means parseGitHubRemote performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without parseGitHubRemote, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

trimmed: trimmed stores a computed value for temporary calculation. It is initialized from `String(remoteUrl || "").trim()`. Within the function it is accesses properties/methods match, replace; passed into URL; assigned or updated later in the function.

sshMatch: sshMatch stores a computed value for temporary calculation. It is initialized from `trimmed.match(/^git@github\.com:([^\s]+)\.(\.+)?(?:\.git)?$/i)`. Within the function it is assigned or updated later in the function.

parsed: parsed stores a constructed object for temporary calculation. It is initialized from `new URL(trimmed.replace(/^git\+/, "").replace(/:/g, "%3A"))`. Within the function it is accesses properties/methods hostname, pathname; assigned or updated later in the function.

parts: parts stores a computed value for temporary calculation. It is initialized from `parsed.pathname.replace(/^\/+/, "").split("/")`. Within the function it is accesses properties/methods length; assigned or updated later in the function.

async function authenticateGitHubForTarget(options = {})

Purpose and intuition: authenticateGitHubForTarget is the interactive setup path for a publishing target. It validates the target remote, initializes or fixes the local Git repository when necessary, stores credentials when credentials are supplied, verifies write access, saves the target configuration, and records the authorization cache. It is the correct first step before Load from target or Apply to site.

Purpose and intuition: The function exists because merely typing a GitHub URL does not prove the builder can read from or write to that repository. Authentication must be completed, verified, and remembered before the target is considered usable.

Parameters and data it receives: options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: It works with validatePublishRemoteUrl, validateCustomDomain, validateCredentialPair, publishAccessError, getPublishTargetInfo, ensureGitRepository, capturePublishTargetState, setOriginRemote. Those calls show that authenticateGitHubForTarget is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without authenticateGitHubForTarget, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

customDomainProvided: customDomainProvided stores a computed value for temporary calculation. It is initialized from Object.prototype.hasOwnProperty.call(options, "customDomain"). Within the function it is passed into writeTargetCustomDomain; assigned or updated later in the function.

remoteUrl: remoteUrl stores a computed value for network or routing value. It is initialized from validatePublishRemoteUrl(repositoryUrl). Within the function it is passed into detectRemoteDefaultBranch, setOriginRemote, storeGitCredentials; assigned or updated later in the function.

domain: domain stores a computed value for temporary calculation. It is initialized from validateCustomDomain(customDomain). Within the function it is passed into writeTargetCustomDomain; assigned or updated later in the function.

credentials: credentials stores a computed value for Git publishing and authorization state. It is initialized from validateCredentialPair(authUsername, authPassword). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods cleanUsername; assigned or updated later in the function.

snapshot: snapshot stores the completed result of an awaited operation. It is initialized from await capturePublishTargetState(). Within the function it is passed into restorePublishTargetState; assigned or updated later in the function.

status: status stores the completed result of an awaited operation. It is initialized from await getGitStatus(). Within the function it is accesses properties/methods credentialManager, git; passed into publishAccessError; assigned or updated later in the function.

preliminaryTarget: preliminaryTarget stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is passed into publishAccessError; assigned or updated later in the function.

targetBranch: targetBranch stores the completed result of an awaited operation. It is initialized from await detectRemoteDefaultBranch(remoteUrl) || "main". It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into checkoutPublishBranch; assigned or updated later in the function.

targetBeforeAuth: targetBeforeAuth stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods branch, remote, repository; passed into publishAccessError; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from await readPublishAuthCache(). Within the function it is accesses properties/methods checkedAt, extendedTrust, successCountLastWeek, trustDays; passed into Boolean, publishAuthCacheExpiresAt; assigned or updated later in the function.

cachedAccess: cachedAccess stores a structured object for runtime state or cache. It is initialized from {. Within the function it is accesses properties/methods branch; assigned or updated later in the function.

target: target stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

storedCredentials: storedCredentials stores the completed result of an awaited operation. It is initialized from await storeGitCredentials(remoteUrl, authUsername, authPassword). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods stored, username; assigned or updated later in the function.

access: access stores the completed result of an awaited operation. It is initialized from await assertPublishAccess({ force: true, requireCompatible: false }). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

target: target stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

AI backend

The functions in this group all support ai backend. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function cleanConversationHistory(value)

Purpose and intuition: cleanConversationHistory belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `clampText`. Those calls show that `cleanConversationHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `cleanConversationHistory`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function extractOpenAiText(data)

Purpose and intuition: `extractOpenAiText` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `data` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `extractOpenAiText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `extractOpenAiText`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

output: `output` stores a computed value for temporary calculation. It is initialized from `Array.isArray(data?.output) ? data.output : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; passed into `isArray`; assigned or updated later in the function.

chunks: `chunks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

function extractOllamaText(data = {})

Purpose and intuition: `extractOllamaText` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `data` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means `extractOllamaText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `extractOllamaText`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })

Purpose and intuition: `callOllamaPortfolioAi` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `conversation` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. `allowWebSearch` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it waits for asynchronous work, calls a network resource and uses a timer to avoid blocking or to wait for another process. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: serializes structured data before sending or saving it; performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with `clampText`, `extractOllamaText`. Those calls show that `callOllamaPortfolioAi` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `callOllamaPortfolioAi`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

host: `host` stores a environment-derived value for temporary calculation. It is initialized from `process.env.OLLAMA_HOST` || `"http://127.0.0.1:11434"`. Within the function it is accesses properties/methods replace; passed into `fetch`; assigned or updated later in the function.

model: `model` stores a environment-derived value for temporary calculation. It is initialized from `process.env.OLLAMA_MODEL` || `"llama3.2"`. Within the function it is passed into `stringify`; assigned or updated later in the function.

controller: controller stores a constructed object for temporary calculation. It is initialized from new AbortController(). Within the function it is accesses properties/methods abort, signal; assigned or updated later in the function.

timeout: timeout stores a function or callback value for temporary calculation. It is initialized from setTimeout(() => controller.abort(), Number(process.env.OLLAMA_TIMEOUT_MS || 45000)). Within the function it is passed into clearTimeout; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from await fetch(`\${host.replace(/\/+\$/, "")}/api/chat`, {). Within the function it is accesses properties/methods json, ok, status; assigned or updated later in the function.

data: data stores a function or callback value for temporary calculation. It is initialized from await response.json().catch(() => ({})). Within the function it is passed into extractOllamaText; assigned or updated later in the function.

answer: answer stores a computed value for temporary calculation. It is initialized from extractOllamaText(data). Within the function it is returned to the caller; assigned or updated later in the function.

function ruleBasedConversationAnswer(question = "")

Purpose and intuition: ruleBasedConversationAnswer belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means ruleBasedConversationAnswer performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without ruleBasedConversationAnswer, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from String(question || "").toLowerCase(). Within the function it is passed into test; assigned or updated later in the function.

async function handlePortfolioAi(request, response)

Purpose and intuition: handlePortfolioAi is the backend side of Ask My Portfolio. It reads the visitor's question, conversation history, intent, and context, enriches that context when public sources are allowed, then chooses an AI provider path. It can call OpenAI when configured, call Ollama locally when available, or fall back to rule-based answers when no model is available.

Purpose and intuition: The function matters because the public script should not hold private API keys or perform privileged source fetching directly. It also gives the assistant one place to enforce prompt rules about portfolio-specific answers, general engineering answers, public source usage, and uncertainty.

Parameters and data it receives: request is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. response is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

What it does: Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: serializes structured data before sending or saving it; performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with readRequestJson, clampText, cleanConversationHistory, sendJson, enrichPortfolioContext, callOllamaPortfolioAi, extractOpenAiText. Those calls show that handlePortfolioAi is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without handlePortfolioAi, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

body: body stores the completed result of an awaited operation. It is initialized from await readRequestJson(request). Within the function it is accesses properties/methods allowWebSearch, context, conversation, intent, question; passed into clampText, cleanConversationHistory, enrichPortfolioContext, fetch, has; assigned or updated later in the function.

question: question stores a computed value for temporary calculation. It is initialized from clampText(body.question, 1200).trim(). Within the function it is passed into callOllamaPortfolioAi, clampText; assigned or updated later in the function.

conversation: conversation stores a computed value for temporary calculation. It is initialized from cleanConversationHistory(body.conversation). Within the function it is passed into callOllamaPortfolioAi, clampText, cleanConversationHistory; assigned or updated later in the function.

validIntents: validIntents stores a constructed object for lookup collection. It is initialized from new Set(["general_conversation", "general_engineering", "general_knowledge", "portfolio_specific"]). Within the function it is accesses properties/methods has; assigned or updated later in the function.

intent: intent stores a computed value for temporary calculation. It is initialized from validIntents.has(body.intent) ? body.intent : "portfolio_specific". Within the function it is passed into callOllamaPortfolioAi, has; assigned or updated later in the function.

allowWebSearch: allowWebSearch stores a computed value for temporary calculation. It is initialized from body.allowWebSearch === true. Within the function it is passed into callOllamaPortfolioAi; assigned or updated later in the function.

context: context stores the completed result of an awaited operation. It is initialized from `await enrichPortfolioContext({ ...(body.context || {}), question })`. Within the function it is passed into `callOllamaPortfolioAi`, `clampText`, `enrichPortfolioContext`; assigned or updated later in the function.

apiKey: apiKey stores a environment-derived value for temporary calculation. It is initialized from `process.env.OPENAI_API_KEY || ""`. Within the function it is passed into `fetch`; assigned or updated later in the function.

ollama: ollama stores the completed result of an awaited operation. It is initialized from `await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })`. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Within the function it is accesses properties/methods `answer`, `error`, `model`, `ok`; passed into `sendJson`; assigned or updated later in the function.

model: model stores a environment-derived value for temporary calculation. It is initialized from `process.env.OPENAI_MODEL || "gpt-5.4"`. Within the function it is passed into `callModel`, `sendJson`; assigned or updated later in the function.

fallbackModel: fallbackModel stores a environment-derived value for temporary calculation. It is initialized from `process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1"`. Within the function it is passed into `callModel`; assigned or updated later in the function.

webSearchMode: webSearchMode stores a environment-derived value for temporary calculation. It is initialized from `String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase()`. Within the function it is assigned or updated later in the function.

enableWebSearch: enableWebSearch stores a computed value for temporary calculation. It is initialized from `webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch)`. Within the function it is assigned or updated later in the function.

buildPayload: buildPayload stores a function or callback value for temporary calculation. It is initialized from `(selectedModel) => {}`. Within the function it is assigned or updated later in the function.

callModel: callModel stores a function or callback value for temporary calculation. It is initialized from `async (selectedModel) => {}`. Within the function it is assigned or updated later in the function.

payload: payload stores a computed value for temporary calculation. It is initialized from `buildPayload(selectedModel)`. Within the function it is accesses properties/methods `tools`; passed into `fetch`; assigned or updated later in the function.

openAiResponse: openAiResponse stores data returned from a network call. It is initialized from `await fetch("https://api.openai.com/v1/responses", {})`. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Within the function it is accesses properties/methods `json`, `ok`, `status`; passed into `sendJson`; assigned or updated later in the function.

data: data stores a function or callback value for temporary calculation. It is initialized from `await openAiResponse.json().catch(() => ({}))`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

destructured { data, openAiResponse, selectedModel }: destructured { data, openAiResponse, selectedModel } stores the completed result of an awaited operation. It is initialized from `await callModel(model)`. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection.

Publishing and GitHub authorization

The functions in this group all support publishing and github authorization. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function publishAccessError(message, details = "", extra = {})

Purpose and intuition: publishAccessError belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: message is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. details is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. extra is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means publishAccessError performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without publishAccessError, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

error: error stores a constructed object for temporary calculation. It is initialized from new Error(message). Within the function it is returned to the caller; accesses properties/methods code, details, publishAccess; assigned or updated later in the function.

function gitFailureText(error)

Purpose and intuition: gitFailureText belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: error is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means gitFailureText performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `gitFailureText`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `remoteUrlForDisplay(remoteUrl = "")`

Purpose and intuition: `remoteUrlForDisplay` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `remoteUrlForDisplay` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `remoteUrlForDisplay`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `validatePublishRemoteUrl(value = "")`

Purpose and intuition: `validatePublishRemoteUrl` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: No other named helper from this file is detected inside the body. That means `validatePublishRemoteUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `validatePublishRemoteUrl`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

remoteUrl: remoteUrl stores a computed value for network or routing value. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; passed into `URL`, `test`; assigned or updated later in the function.

parsed: parsed stores a constructed object for temporary calculation. It is initialized from `new URL(remoteUrl)`. Within the function it is accesses properties/methods `hostname`, `pathname`, `protocol`; assigned or updated later in the function.

parts: parts stores a computed value for temporary calculation. It is initialized from `parsed.pathname.replace(/^V+/, "").split("/")`. Within the function it is accesses properties/methods `length`; assigned or updated later in the function.

function validateCustomDomain(value = "")

Purpose and intuition: validateCustomDomain is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: No other named helper from this file is detected inside the body. That means validateCustomDomain performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without validateCustomDomain, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

domain: domain stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase()`. Within the function it is returned to the caller; passed into `Error`, `test`; assigned or updated later in the function.

async function bumpPublishedAssetVersions(baseRoot = portfolioRoot)

Purpose and intuition: bumpPublishedAssetVersions belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: baseRoot is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; writes data to disk as part of the operation; reads data from disk as part of the operation.

What it calls or works with: No other named helper from this file is detected inside the body. That means bumpPublishedAssetVersions performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without bumpPublishedAssetVersions, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

html: html stores a string value for temporary calculation. It is initialized from `""`. Within the function it is passed into join; assigned or updated later in the function.

indexPath: indexPath stores a resolved filesystem or route value. It is initialized from `path.join(baseRoot, "index.html")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `readFile`, `writeFile`; assigned or updated later in the function.

version: version stores a computed value for temporary calculation. It is initialized from `Date.now().toString()`. Within the function it is assigned or updated later in the function.

next: next stores a computed value for temporary calculation. It is initialized from `html`. Within the function it is passed into `writeFile`; assigned or updated later in the function.

async function getPublishTargetInfo()

Purpose and intuition: getPublishTargetInfo belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: reads data from disk as part of the operation.

What it calls or works with: It works with `samePath`, `workspaceHasCompatibleSiteFiles`, `runPublishGit`, `remoteUrlForDisplay`, `parseGitHubRemote`, `resolveInsidePortfolioRoot`, `readPublishAuthCache`, `publishAuthCachelsFresh`. Those calls show that getPublishTargetInfo is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `getPublishTargetInfo`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

info: `info` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `authorizationCached`, `authorizationChecked`, `branch`, `checkedAt`, `customDomain`, `expiresAt`, `extendedTrust`, `gitBacked`; passed into `parseGitHubRemote`; assigned or updated later in the function.

parsedRemote: `parsedRemote` stores a computed value for network or routing value. It is initialized from `parseGitHubRemote(info.remote)`. Within the function it is accesses properties/methods `owner`, `repo`; assigned or updated later in the function.

accessShape: `accessShape` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is assigned or updated later in the function.

cached: `cached` stores the completed result of an awaited operation. It is initialized from `await readPublishAuthCache()`. Within the function it is accesses properties/methods `checkedAt`, `extendedTrust`, `successCountLastWeek`, `trustDays`; passed into `Boolean`, `publishAuthCacheExpiresAt`; assigned or updated later in the function.

async function runGit(args, options = {})

Purpose and intuition: `runGit` executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

Parameters and data it receives: `args` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: No other named helper from this file is detected inside the body. That means `runGit` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `runGit`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

lastError: `lastError` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is assigned or updated later in the function.

candidate: `candidate` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into `execFileAsync`.

result: result stores the completed result of an awaited operation. It is initialized from `await execFileAsync(candidate, args, {`. Within the function it is assigned or updated later in the function.

async function runPublishGit(args, options = {})

Purpose and intuition: `runPublishGit` executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

Parameters and data it receives: `args` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `runGit`. Those calls show that `runPublishGit` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `runPublishGit`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function getGitStatus()

Purpose and intuition: `getGitStatus` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `runGit`. Those calls show that `getGitStatus` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `getGitStatus`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

git: git stores a structured object for Git publishing and authorization state. It is initialized from { ok: false, stdout: "", stderr: "", error: "Git for Windows was not found." }. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods error, ok, stderr, stdout; assigned or updated later in the function.

gitResult: gitResult stores the completed result of an awaited operation. It is initialized from await runGit(["--version"]). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods stderr, stdout; assigned or updated later in the function.

credentialManager: credentialManager stores a structured object for Git publishing and authorization state. It is initialized from { ok: false, version: "", error: "Git Credential Manager was not found." }. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

gcm: gcm stores a structured object for temporary calculation. It is initialized from { ok: false, stdout: "", stderr: "", error: "Git Credential Manager was not found." }. Within the function it is accesses properties/methods error, ok, stderr, stdout; assigned or updated later in the function.

gcmResult: gcmResult stores the completed result of an awaited operation. It is initialized from await runGit(["credential-manager", "--version"]). Within the function it is accesses properties/methods stderr, stdout; assigned or updated later in the function.

async function publishPathsStageable(relativePath)

Purpose and intuition: publishPathsStageable belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: relativePath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with resolveInsidePortfolioRoot, runPublishGit. Those calls show that publishPathsStageable is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without publishPathsStageable, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function stageablePublishPaths(pathsToCheck)

Purpose and intuition: stageablePublishPaths belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `pathsToCheck` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `publishPathsStageable`. Those calls show that `stageablePublishPaths` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `stageablePublishPaths`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

stageable: `stageable` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

relativePath: `relativePath` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`.

async function `runGitWithInput(args, input, options = {})`

Purpose and intuition: `runGitWithInput` executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

Parameters and data it receives: `args` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `input` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and runs an external command. It returns the awaited result of another asynchronous operation, so its caller receives the completed value rather than a pending `Promise`. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: No other named helper from this file is detected inside the body. That means `runGitWithInput` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `runGitWithInput`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

lastError: lastError stores a computed value for temporary calculation. It is initialized from null. Within the function it is assigned or updated later in the function.

candidate: candidate starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into resolve, spawn.

child: child stores a computed value for temporary calculation. It is initialized from spawn(candidate, args, {). Within the function it is accesses properties/methods once, stderr, stdin, stdout; assigned or updated later in the function.

stdout: stdout stores a string value for temporary calculation. It is initialized from "". Within the function it is accesses properties/methods on; passed into Error, resolve; assigned or updated later in the function.

stderr: stderr stores a string value for temporary calculation. It is initialized from "". Within the function it is accesses properties/methods on; passed into Error, resolve; assigned or updated later in the function.

error: error stores a constructed object for temporary calculation. It is initialized from new Error(stderr || stdout || `Git exited with code \${code}.`). Within the function it is accesses properties/methods code, git, stderr, stdout; passed into catch, once, reject; assigned or updated later in the function.

async function storeGitCredentials(remoteUrl, username, password)

Purpose and intuition: storeGitCredentials belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: remoteUrl points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. username is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. password is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: It works with runPublishGit, runGitWithInput. Those calls show that storeGitCredentials is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without storeGitCredentials, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

cleanUsername: cleanUsername stores a computed value for temporary calculation. It is initialized from String(username || "").trim(). Within the function it is assigned or updated later in the function.

cleanPassword: cleanPassword stores a computed value for temporary calculation. It is initialized from `String(password || "")`. Within the function it is assigned or updated later in the function.

parsed: parsed stores a computed value for temporary calculation. It is initialized from null. Within the function it accesses properties/methods `hostname`, `pathname`, `protocol`; assigned or updated later in the function.

pathName: pathName stores a computed value for filesystem location. It is initialized from `parsed.pathname.replace(/^\/+/, "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is assigned or updated later in the function.

credentialInput: credentialInput stores a list for Git publishing and authorization state. It is initialized from `[]`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runGitWithInput`; assigned or updated later in the function.

function validateCredentialPair(username, password)

Purpose and intuition: validateCredentialPair is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: username is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. password is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: No other named helper from this file is detected inside the body. That means validateCredentialPair performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without validateCredentialPair, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

cleanUsername: cleanUsername stores a computed value for temporary calculation. It is initialized from `String(username || "").trim()`. Within the function it is assigned or updated later in the function.

cleanPassword: cleanPassword stores a computed value for temporary calculation. It is initialized from `String(password || "")`. Within the function it is assigned or updated later in the function.

function normalizeGitBranchName(value = "")

Purpose and intuition: normalizeGitBranchName standardizes GitBranchName data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means `normalizeGitBranchName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `normalizeGitBranchName`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

branch: branch stores a computed value for Git publishing and authorization state. It is initialized from `String(value || "").trim()`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; accesses properties/methods `endsWith`, `includes`, `length`, `startsWith`; passed into `test`; assigned or updated later in the function.

async function detectRemoteDefaultBranch(remoteUrl = "")

Purpose and intuition: `detectRemoteDefaultBranch` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Parameters and data it receives: `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `runPublishGit`, `normalizeGitBranchName`. Those calls show that `detectRemoteDefaultBranch` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `detectRemoteDefaultBranch`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

result: result stores the completed result of an awaited operation. It is initialized from `await runPublishGit(["Is-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 })`. Within the function it is accesses properties/methods `stdout`; passed into `String`; assigned or updated later in the function.

match: match stores a computed value for temporary calculation. It is initialized from `String(result.stdout || "").match(/^ref:\s+refs\heads\(.+\)\s+HEAD$/m)`. Within the function it is passed into `normalizeGitBranchName`; assigned or updated later in the function.

async function originRemoteExists()

Purpose and intuition: `originRemoteExists` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `runPublishGit`. Those calls show that `originRemoteExists` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `originRemoteExists`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function setOriginRemote(remoteUrl)

Purpose and intuition: `setOriginRemote` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it waits for asynchronous work. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `originRemoteExists`, `runPublishGit`. Those calls show that `setOriginRemote` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `setOriginRemote`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function checkoutPublishBranch(branch)

Purpose and intuition: checkoutPublishBranch belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: branch is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with normalizeGitBranchName, runPublishGit. Those calls show that checkoutPublishBranch is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without checkoutPublishBranch, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

cleanBranch: cleanBranch stores a computed value for Git publishing and authorization state. It is initialized from normalizeGitBranchName(branch) || "main". It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; passed into runPublishGit; assigned or updated later in the function.

current: current stores a function or callback value for temporary calculation. It is initialized from (await runPublishGit(["branch", "--show-current"])).stdout.trim(). Within the function it is passed into runPublishGit; assigned or updated later in the function.

async function verifyPublishWriteAccess(access = {})

Purpose and intuition: verifyPublishWriteAccess belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: access is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: It works with `safeSegment`, `runPublishGit`, `publishAccessError`, `gitFailureText`. Those calls show that `verifyPublishWriteAccess` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `verifyPublishWriteAccess`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

checkBranch: `checkBranch` stores a string value for Git publishing and authorization state. It is initialized from ``omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}``. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

async function ensurePublishHeadForWriteCheck()

Purpose and intuition: `ensurePublishHeadForWriteCheck` creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `runPublishGit`. Those calls show that `ensurePublishHeadForWriteCheck` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `ensurePublishHeadForWriteCheck`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function synchronizePublishBranchFromRemote(branch, access = {})

Purpose and intuition: `synchronizePublishBranchFromRemote` reconciles two states that can drift apart. In this project, sync functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

Parameters and data it receives: `branch` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `access` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers

must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: It works with `normalizeGitBranchName`, `runPublishGit`, `checkoutPublishBranch`, `publishAccessError`, `gitFailureText`. Those calls show that `synchronizePublishBranchFromRemote` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `synchronizePublishBranchFromRemote`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

cleanBranch: `cleanBranch` stores a computed value for Git publishing and authorization state. It is initialized from `normalizeGitBranchName(branch) || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `checkoutPublishBranch`, `runPublishGit`; assigned or updated later in the function.

remoteBranch: `remoteBranch` stores the completed result of an awaited operation. It is initialized from `await runPublishGit(["ls-remote", "--heads", "origin", cleanBranch], {`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods `stdout`; assigned or updated later in the function.

async function capturePublishTargetState()

Purpose and intuition: `capturePublishTargetState` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: reads data from disk as part of the operation.

What it calls or works with: It works with `runPublishGit`, `resolveInsidePortfolioRoot`. Those calls show that `capturePublishTargetState` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `capturePublishTargetState`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

snapshot: snapshot stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `branch`, `customDomainExists`, `customDomainText`, `remote`, `remoteExists`; assigned or updated later in the function.

async function restorePublishTargetState(snapshot = {})

Purpose and intuition: `restorePublishTargetState` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `snapshot` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: writes data to disk as part of the operation.

What it calls or works with: It works with `setOriginRemote`, `originRemoteExists`, `runPublishGit`, `checkoutPublishBranch`, `resolveInsidePortfolioRoot`. Those calls show that `restorePublishTargetState` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `restorePublishTargetState`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function writeTargetCustomDomain(domain, customDomainProvided)

Purpose and intuition: `writeTargetCustomDomain` writes `TargetCustomDomain` to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

Parameters and data it receives: `domain` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `customDomainProvided` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: writes data to disk as part of the operation.

What it calls or works with: It works with `resolveInsidePortfolioRoot`, `samePath`, `resolveInsideRoot`. Those calls show that `writeTargetCustomDomain` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `writeTargetCustomDomain`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

targetPaths: `targetPaths` stores a list for filesystem location. It is initialized from `[resolveInsidePortfolioRoot("CNAME")]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

targetPath: `targetPath` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `rm`, `writeFile`.

targetPath: `targetPath` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `rm`, `writeFile`.

async function ensureGitRepository()

Purpose and intuition: `ensureGitRepository` creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: creates folders before writing files so missing directories do not crash the workflow.

What it calls or works with: It works with `runPublishGit`. Those calls show that `ensureGitRepository` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `ensureGitRepository`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

insideWorkTree: insideWorkTree stores a function or callback value for temporary calculation. It is initialized from (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim(). Within the function it is assigned or updated later in the function.

branch: branch stores a function or callback value for Git publishing and authorization state. It is initialized from (await runPublishGit(["branch", "--show-current"])).stdout.trim(). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into runPublishGit; assigned or updated later in the function.

async function configurePublishTarget(options = {})

Purpose and intuition: configurePublishTarget belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with validatePublishRemoteUrl, validateCustomDomain, validateCredentialPair, ensureGitRepository, setOriginRemote, detectRemoteDefaultBranch, checkoutPublishBranch, writeTargetCustomDomain. Those calls show that configurePublishTarget is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without configurePublishTarget, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

customDomainProvided: customDomainProvided stores a computed value for temporary calculation. It is initialized from Object.prototype.hasOwnProperty.call(options, "customDomain"). Within the function it is passed into writeTargetCustomDomain; assigned or updated later in the function.

remoteUrl: remoteUrl stores a computed value for network or routing value. It is initialized from validatePublishRemoteUrl(repositoryUrl). Within the function it is passed into detectRemoteDefaultBranch, setOriginRemote; assigned or updated later in the function.

domain: domain stores a computed value for temporary calculation. It is initialized from validateCustomDomain(customDomain). Within the function it is passed into writeTargetCustomDomain; assigned or updated later in the function.

defaultBranch: defaultBranch stores the completed result of an awaited operation. It is initialized from await detectRemoteDefaultBranch(remoteUrl). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into checkoutPublishBranch; assigned or updated later in the function.

currentRemote: currentRemote stores the completed result of an awaited operation. It is initialized from `remoteUrl || (await getPublishTargetInfo()).remote`. Within the function it is passed into `storeGitCredentials`; assigned or updated later in the function.

credentials: credentials stores the completed result of an awaited operation. It is initialized from `await storeGitCredentials(currentRemote, authUsername, authPassword)`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods stored, username; assigned or updated later in the function.

target: target stores the completed result of an awaited operation. It is initialized from `await getPublishTargetInfo()`. Within the function it is assigned or updated later in the function.

async function readPublishAuthCache()

Purpose and intuition: `readPublishAuthCache` reads `PublishAuthCache` from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: parses JSON rather than treating incoming text as already trusted data; reads data from disk as part of the operation.

What it calls or works with: No other named helper from this file is detected inside the body. That means `readPublishAuthCache` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `readPublishAuthCache`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function writePublishAuthCache(access = {})

Purpose and intuition: `writePublishAuthCache` writes `PublishAuthCache` to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

Parameters and data it receives: `access` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: serializes structured data before sending or saving it; writes data to disk as part of the operation.

What it calls or works with: It works with `readPublishAuthCache`, `parsePublishAuthTimestamp`, `publishAuthCacheScope`. Those calls show that `writePublishAuthCache` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `writePublishAuthCache`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

previousCache: `previousCache` stores the completed result of an awaited operation. It is initialized from `await readPublishAuthCache()`. Within the function it is accesses properties/methods `branch`, `remote`, `repository`, `successHistory`; passed into `isArray`; assigned or updated later in the function.

checkedAt: `checkedAt` stores a constructed object for temporary calculation. It is initialized from `new Date()`. Within the function it is accesses properties/methods `getTime`, `toISOString`; assigned or updated later in the function.

checkedAtMs: `checkedAtMs` stores a computed value for temporary calculation. It is initialized from `checkedAt.getTime()`. Within the function it is passed into `Date`; assigned or updated later in the function.

sameTarget: `sameTarget` stores a computed value for temporary calculation. It is initialized from `previousCache`. Within the function it is assigned or updated later in the function.

previousHistory: `previousHistory` stores a computed value for runtime state or cache. It is initialized from `sameTarget && Array.isArray(previousCache.successHistory)`. Within the function it is assigned or updated later in the function.

successTimestamps: `successTimestamps` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

successHistory: `successHistory` stores a function or callback value for runtime state or cache. It is initialized from `successTimestamps.map((timestamp) => new Date(timestamp).toISOString())`. Within the function it is passed into `isArray`; assigned or updated later in the function.

successCountLastWeek: `successCountLastWeek` stores a computed value for temporary calculation. It is initialized from `successTimestamps`. Within the function it is assigned or updated later in the function.

extendedTrust: `extendedTrust` stores a computed value for temporary calculation. It is initialized from `successCountLastWeek > publishAuthExtendedThreshold`. Within the function it is assigned or updated later in the function.

ttlMs: `ttlMs` stores a computed value for temporary calculation. It is initialized from `extendedTrust ? publishAuthExtendedTtlMs : publishAuthCacheTtlMs`. Within the function it is passed into `Date`; assigned or updated later in the function.

cache: `cache` stores a structured object for runtime state or cache. It is initialized from `{}`. Within the function it is returned to the caller; passed into `writeFile`; assigned or updated later in the function.

function publishAuthCacheScope()

Purpose and intuition: publishAuthCacheScope belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means publishAuthCacheScope performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without publishAuthCacheScope, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

username: username stores a environment-derived value for temporary calculation. It is initialized from `process.env.USERNAME || process.env.USER || ""`. Within the function it is assigned or updated later in the function.

function parsePublishAuthTimestamp(value = "")

Purpose and intuition: parsePublishAuthTimestamp interprets PublishAuthTimestamp text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means parsePublishAuthTimestamp performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without parsePublishAuthTimestamp, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

direct: direct stores a computed value for filesystem location. It is initialized from `Date.parse(String(value || ""))`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews,

compiler artifacts, or published assets. Within the function it is returned to the caller; assigned or updated later in the function.

normalized: normalized stores a computed value for temporary calculation. It is initialized from `String(value || "").replace(/(\.d{3})d+(Z|[-+]d{2}:\d{2})$/i, "$1$2")`. Within the function it is passed into `parse`; assigned or updated later in the function.

function publishAuthCacheExpiresAt(cache)

Purpose and intuition: `publishAuthCacheExpiresAt` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `cache` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `parsePublishAuthTimestamp`. Those calls show that `publishAuthCacheExpiresAt` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `publishAuthCacheExpiresAt`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

explicitExpiresAt: `explicitExpiresAt` stores a computed value for temporary calculation. It is initialized from `parsePublishAuthTimestamp(cache?.expiresAt || "")`. Within the function it is passed into `Date`; assigned or updated later in the function.

checkedAt: `checkedAt` stores a computed value for temporary calculation. It is initialized from `parsePublishAuthTimestamp(cache?.checkedAt || "")`. Within the function it is passed into `Date`, `parsePublishAuthTimestamp`; assigned or updated later in the function.

function publishAuthCachelsFresh(cache, access)

Purpose and intuition: `publishAuthCachelsFresh` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `cache` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `access` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `publishAuthCacheScope`, `publishAuthCacheExpiresAt`. Those calls show that `publishAuthCachelsFresh` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `publishAuthCachelsFresh`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

expiresAt: `expiresAt` stores a computed value for temporary calculation. It is initialized from `Date.parse(publishAuthCacheExpiresAt(cache))`. Within the function it is assigned or updated later in the function.

async function assertPublishAccess(options = {})

Purpose and intuition: `assertPublishAccess` is the gatekeeper that decides whether the current machine and target are allowed to change the website. It checks the configured publishing target, compares cached authorization against the current remote and branch, respects the once-per-day authentication window, and can force a fresh verification when needed. It returns the access object that publishing functions use as proof.

Purpose and intuition: Without this function, Apply to site would either ask for GitHub authorization too often or push too freely. The cache logic is what makes the app usable day to day while still preventing an unauthenticated installation from silently modifying Maurice's website.

Parameters and data it receives: `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: It works with `runPublishGit`, `publishAccessError`, `gitFailureText`, `remoteUrlForDisplay`, `workspaceHasCompatibleSiteFiles`, `parseGitHubRemote`, `readPublishAuthCache`, `publishAuthCachelsFresh`. Those calls show that `assertPublishAccess` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assertPublishAccess`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

insideWorkTree: insideWorkTree stores a string value for temporary calculation. It is initialized from "". Within the function it is assigned or updated later in the function.

remote: remote stores a string value for network or routing value. It is initialized from "". Within the function it is passed into parseGitHubRemote, publishAccessError, remoteUrlForDisplay, runPublishGit; assigned or updated later in the function.

branch: branch stores a string value for Git publishing and authorization state. It is initialized from "". It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into publishAccessError, runPublishGit, synchronizePublishBranchFromRemote; assigned or updated later in the function.

parsedRemote: parsedRemote stores a computed value for network or routing value. It is initialized from parseGitHubRemote(remote). Within the function it is accesses properties/methods owner, repo; assigned or updated later in the function.

repository: repository stores a computed value for temporary calculation. It is initialized from parsedRemote ? `\${parsedRemote.owner}/\${parsedRemote.repo}` : remoteUrlForDisplay(remote). Within the function it is passed into publishAccessError; assigned or updated later in the function.

access: access stores a structured object for temporary calculation. It is initialized from {}. Within the function it is passed into synchronizePublishBranchFromRemote, verifyPublishWriteAccess, writePublishAuthCache; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from await readPublishAuthCache(). Within the function it is accesses properties/methods checkedAt, extendedTrust, successCountLastWeek, trustDays; passed into Boolean, publishAuthCacheExpiresAt; assigned or updated later in the function.

remoteSync: remoteSync stores the completed result of an awaited operation. It is initialized from await synchronizePublishBranchFromRemote(branch, access). Within the function it is assigned or updated later in the function.

localHead: localHead stores the completed result of an awaited operation. It is initialized from await ensurePublishHeadForWriteCheck(). Within the function it is assigned or updated later in the function.

writeCheck: writeCheck stores the completed result of an awaited operation. It is initialized from await verifyPublishWriteAccess(access). Within the function it is assigned or updated later in the function.

authCache: authCache stores the completed result of an awaited operation. It is initialized from await writePublishAuthCache(access). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods checkedAt, expiresAt, extendedTrust, successCountLastWeek, trustDays; passed into Boolean; assigned or updated later in the function.

async function syncFromPublishTarget()

Purpose and intuition: syncFromPublishTarget is the import path. After a target is authenticated, this function pulls or fetches the latest public portfolio files from the selected repository and copies compatible website assets back into the local builder workspace. It is what lets a fresh machine reconstruct the current portfolio state from the website repository.

Purpose and intuition: This function is deliberately separate from authentication. Authenticate target proves access. Load from target reads the target. Keeping those concepts separate makes the UI clearer and reduces accidental overwrites.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

What it calls or works with: It works with `assertPublishAccess`, `runPublishGit`, `runGit`, `publishAccessError`, `resolveInsideRoot`, `syncPortfolioPublishFiles`. Those calls show that `syncFromPublishTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `syncFromPublishTarget`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

publishAccess: `publishAccess` stores the completed result of an awaited operation. It is initialized from `await assertPublishAccess({ requireCompatible: false })`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods `branch`; passed into `publishAccessError`; assigned or updated later in the function.

branch: `branch` stores a computed value for Git publishing and authorization state. It is initialized from `publishAccess.branch || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runGit`; assigned or updated later in the function.

remote: `remote` stores a function or callback value for network or routing value. It is initialized from `(await runPublishGit(["remote", "get-url", "origin"])).stdout.trim()`. Within the function it is passed into `runGit`, `runPublishGit`; assigned or updated later in the function.

importRoot: `importRoot` stores a resolved filesystem or route value. It is initialized from `await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"))`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `rm`, `runGit`; assigned or updated later in the function.

cloneRoot: `cloneRoot` stores a resolved filesystem or route value. It is initialized from `path.join(importRoot, "target")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `fsAccess`, `join`, `readFile`, `runGit`; assigned or updated later in the function.

importPaths: `importPaths` stores a list for filesystem location. It is initialized from `[]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is assigned or updated later in the function.

availablePaths: `availablePaths` stores a list for filesystem location. It is initialized from `[]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or

published assets. Within the function it is mutated as a collection or DOM container; accesses properties/methods includes, push; assigned or updated later in the function.

importPath: importPath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, fsAccess, join, push, resolveInsideRoot.

backupRoot: backupRoot stores a computed value for filesystem location. It is initialized from resolveInsideRoot(). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, mkdir, relative; assigned or updated later in the function.

importPath: importPath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, fsAccess, join, push, resolveInsideRoot.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from path.join(cloneRoot, importPath). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp; assigned or updated later in the function.

targetPath: targetPath stores a resolved filesystem or route value. It is initialized from resolveInsideRoot(importPath). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, fsAccess, rm; assigned or updated later in the function.

importedCatalog: importedCatalog stores a resolved filesystem or route value. It is initialized from await readFile(path.join(cloneRoot, "projects.json"), "utf8"). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into writeFile; assigned or updated later in the function.

portfolioSync: portfolioSync stores the completed result of an awaited operation. It is initialized from await syncPortfolioPublishFiles({ removeMissing: false }). Within the function it is assigned or updated later in the function.

async function installGitForWindows()

Purpose and intuition: installGitForWindows installs or prepares external tooling. It belongs in the backend because installation touches the machine, not just the browser page.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with runOptionalCommand. Those calls show that installGitForWindows is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `installGitForWindows`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

winget: `winget` stores the completed result of an awaited operation. It is initialized from `await runOptionalCommand("winget", ["--version"])`. Within the function it is accesses properties/methods `ok`; passed into `runOptionalCommand`, `spawn`; assigned or updated later in the function.

downloadUrl: `downloadUrl` stores a string value for network or routing value. It is initialized from `"https://git-scm.com/download/win"`. Within the function it is assigned or updated later in the function.

child: `child` stores a computed value for temporary calculation. It is initialized from `spawn("winget", ["install", "--id", "Git.Git", "-e", "--source", "winget"], {`. Within the function it is accesses properties/methods `unref`; assigned or updated later in the function.

async function syncPortfolioPublishFiles(options = {})

Purpose and intuition: `syncPortfolioPublishFiles` reconciles two states that can drift apart. In this project, `sync` functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

Parameters and data it receives: `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work and reads or writes files. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: creates folders before writing files so missing directories do not crash the workflow.

What it calls or works with: It works with `samePath`, `resolveInsideRoot`, `resolveInsidePortfolioRoot`, `pathExists`. Those calls show that `syncPortfolioPublishFiles` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `syncPortfolioPublishFiles`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

copied: `copied` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

skipped: `skipped` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

relativePath: `relativePath` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file

work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into push, resolveInsidePortfolioRoot, resolveInsideRoot.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from resolveInsideRoot(relativePath). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp; assigned or updated later in the function.

targetPath: targetPath stores a resolved filesystem or route value. It is initialized from resolveInsidePortfolioRoot(relativePath). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, mkdir, rm; assigned or updated later in the function.

async function publishSiteChanges(publishAccess = null)

Purpose and intuition: publishSiteChanges is the final publishing operation behind Apply to site. It assumes authorization has been proven or supplied, synchronizes publishable files into the portfolio mirror, checks Git status, stages only approved paths, creates a commit when there are changes, and pushes to the selected branch. It is async because Git commands and file synchronization are external operations that take time and can fail.

Purpose and intuition: This function is important because publishing is the one place where local draft work becomes public website content. It must therefore be strict. If it staged arbitrary files or pushed without checking state, builder internals, private drafts, or stale branches could leak to the website. The function works with assertPublishAccess, syncPortfolioPublishFiles, getGitStatus, runPublishGit, and stageablePublishPaths.

Parameters and data it receives: publishAccess is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: throws explicit errors when a required safety or compatibility condition fails.

What it calls or works with: It works with assertPublishAccess, runPublishGit, synchronizePublishBranchFromRemote, syncPortfolioPublishFiles, bumpPublishedAssetVersions, stageablePublishPaths. Those calls show that publishSiteChanges is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without publishSiteChanges, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

access: access stores the completed result of an awaited operation. It is initialized from publishAccess || await assertPublishAccess(). Within the function it is accesses properties/methods branch; passed into synchronizePublishBranchFromRemote; assigned or updated later in the function.

branch: branch stores the completed result of an awaited operation. It is initialized from `access.branch || (await runPublishGit(["branch", "--show-current])).stdout.trim()`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runPublishGit`, `synchronizePublishBranchFromRemote`; assigned or updated later in the function.

remoteSync: remoteSync stores the completed result of an awaited operation. It is initialized from `await synchronizePublishBranchFromRemote(branch, access)`. Within the function it is assigned or updated later in the function.

portfolioSync: portfolioSync stores the completed result of an awaited operation. It is initialized from `await syncPortfolioPublishFiles({ removeMissing: true })`. Within the function it is assigned or updated later in the function.

stageablePaths: stageablePaths stores a resolved filesystem or route value. It is initialized from `await stageablePublishPaths(publishPaths)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `length`; passed into `runPublishGit`; assigned or updated later in the function.

status: status stores a resolved filesystem or route value. It is initialized from `await runPublishGit(["status", "--porcelain", "--", ...stageablePaths])`. Within the function it is accesses properties/methods `stdout`; passed into `runPublishGit`; assigned or updated later in the function.

hasChanges: hasChanges stores a computed value for temporary calculation. It is initialized from `status.stdout.trim().length > 0`. Within the function it is assigned or updated later in the function.

commit: commit stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

message: message stores a string value for temporary calculation. It is initialized from ``Update portfolio site ${new Date().toISOString().slice(0, 10)}``. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

pushArgs: pushArgs stores a computed value for temporary calculation. It is initialized from `branch ? ["push", "origin", branch] : ["push"]`. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

push: push stores the completed result of an awaited operation. It is initialized from `await runPublishGit(pushArgs)`. Within the function it is accesses properties/methods `stderr`, `stdout`; assigned or updated later in the function.

Application update and reporting

The functions in this group all support application update and reporting. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function compareVersions(left = "", right = "")

Purpose and intuition: compareVersions belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: left is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. right is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before

returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `compareVersions` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `compareVersions`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

leftParts: `leftParts` stores a function or callback value for lookup collection. It is initialized from `String(left || "0").split(/[.]/).map((part) => Number.parseInt(part, 10) || 0)`. Within the function it is accesses properties/methods `length`; passed into `max`; assigned or updated later in the function.

rightParts: `rightParts` stores a function or callback value for lookup collection. It is initialized from `String(right || "0").split(/[.]/).map((part) => Number.parseInt(part, 10) || 0)`. Within the function it is accesses properties/methods `length`; passed into `max`; assigned or updated later in the function.

length: `length` stores a computed value for temporary calculation. It is initialized from `Math.max(leftParts.length, rightParts.length)`. Within the function it is passed into `max`; assigned or updated later in the function.

index: `index` stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is assigned or updated later in the function.

delta: `delta` stores a function or callback value for temporary calculation. It is initialized from `(leftParts[index] || 0) - (rightParts[index] || 0)`. Within the function it is returned to the caller; assigned or updated later in the function.

async function `getUpdateInfo()`

Purpose and intuition: `getUpdateInfo` belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with `compareVersions`. Those calls show that `getUpdateInfo` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `getUpdateInfo`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

currentVersion: `currentVersion` stores an environment-derived value for temporary calculation. It is initialized from `process.env.OMB_APP_VERSION || packageJson.version || "0.0.0"`. Within the function it is passed into `compareVersions`; assigned or updated later in the function.

owner: `owner` stores a string value for temporary calculation. It is initialized from `"otienomaurice"`. Within the function it is passed into `fetch`; assigned or updated later in the function.

repo: `repo` stores a string value for temporary calculation. It is initialized from `"omb-portfolio-builder"`. Within the function it is passed into `fetch`; assigned or updated later in the function.

response: `response` stores data returned from a network call. It is initialized from `await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, { ... })`. Within the function it accesses properties/methods `json`, `ok`, `status`; passed into `Error`; assigned or updated later in the function.

release: `release` stores the completed result of an awaited operation. It is initialized from `await response.json()`. Within the function it accesses properties/methods `assets`, `html_url`, `tag_name`; passed into `String`; assigned or updated later in the function.

latestVersion: `latestVersion` stores a computed value for temporary calculation. It is initialized from `String(release.tag_name || "").replace(/^builder-v/i, "")`. Within the function it is passed into `compareVersions`, `has`; assigned or updated later in the function.

installer: `installer` stores a function or callback value for temporary calculation. It is initialized from `(release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name))`. Within the function it is assigned or updated later in the function.

portable: `portable` stores a function or callback value for temporary calculation. It is initialized from `(release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name))`. Within the function it is assigned or updated later in the function.

updateBlocked: `updateBlocked` stores a computed value for temporary calculation. It is initialized from `blockedAppUpdateVersions.has(latestVersion)`. Within the function it is assigned or updated later in the function.

async function getBuilderReleaseDownloadReport()

Purpose and intuition: `getBuilderReleaseDownloadReport` belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must `await` it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure.

What it calls or works with: No other named helper from this file is detected inside the body. That means `getBuilderReleaseDownloadReport` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `getBuilderReleaseDownloadReport`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

owner: owner stores a string value for temporary calculation. It is initialized from "otienomaurice". Within the function it is passed into fetch; assigned or updated later in the function.

repo: repo stores a string value for temporary calculation. It is initialized from "omb-portfolio-builder". Within the function it is passed into fetch; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {`). Within the function it is accesses properties/methods json, ok, status; passed into Error; assigned or updated later in the function.`

release: release stores the completed result of an awaited operation. It is initialized from `await response.json()`. Within the function it is accesses properties/methods assets, html_url, name, tag_name; assigned or updated later in the function.

assets: assets stores a function or callback value for lookup collection. It is initialized from `(release.assets || []).map((asset) => ({`). Within the function it is iterated or transformed as a collection; accesses properties/methods reduce; assigned or updated later in the function.`

async function `getSecurityReport()`

Purpose and intuition: `getSecurityReport` belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `getBuilderReleaseDownloadReport`, `readPublishAuthCache`, `getPublishTargetInfo`, `publishAuthCachesFresh`, `publishAuthCacheExpiresAt`. Those calls show that `getSecurityReport` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `getSecurityReport`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

destructured [downloadResult, authCacheResult, targetResult]: destructured [downloadResult, authCacheResult, targetResult] stores the completed result of an awaited operation. It is initialized from `await Promise.allSettled([`. It affects publishing, where mistakes can change the public website or expose the wrong files.

authCache: authCache stores a computed value for runtime state or cache. It is initialized from `authCacheResult.status === "fulfilled" ? authCacheResult.value : null`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into Boolean, Number, `publishAuthCacheExpiresAt`; assigned or updated later in the function.

target: target stores a computed value for temporary calculation. It is initialized from `targetResult.status === "fulfilled" ? targetResult.value : null`. Within the function it is passed into Boolean; assigned or updated later in the function.

function safeUpdateFileSegment(value = "")

Purpose and intuition: safeUpdateFileSegment is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means safeUpdateFileSegment performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without safeUpdateFileSegment, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

async function downloadAndLaunchAppUpdate()

Purpose and intuition: downloadAndLaunchAppUpdate is the updater handoff. It checks release information, downloads the installer, writes a PowerShell script that can wait for the current process to exit, launches the installer or updater quietly, and schedules the current app to close. It must be async because it performs network download, file writes, and process launches.

Purpose and intuition: The function is needed because a running Electron app cannot safely replace all of its own files while those files are in use. The handoff script becomes a temporary helper that finishes the update after the current process gets out of the way.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it waits for asynchronous work, calls a network resource, runs an external command, reads or writes files and uses a timer to avoid blocking or to wait for another process. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

What it calls or works with: It works with `getUpdateInfo`, `safeUpdateFileSegment`, `powershellSingleQuoted`. Those calls show that `downloadAndLaunchAppUpdate` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `downloadAndLaunchAppUpdate`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

update: update stores the completed result of an awaited operation. It is initialized from `await getUpdateInfo()`. Within the function it is accesses properties/methods `blockedReason`, `currentVersion`, `error`, `installerName`, `installerUrl`, `latestVersion`, `ok`, `updateAvailable`; passed into `Error`, `emit`, `fetch`, `join`, `safeUpdateFileSegment`, `test`; assigned or updated later in the function.

updateRoot: updateRoot stores a resolved filesystem or route value. It is initialized from `path.join(os.tmpdir(), "omb-portfolio-builder-updates")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `mkdir`; assigned or updated later in the function.

installerName: installerName stores a computed value for temporary calculation. It is initialized from `update.installerName && /\.exe$/i.test(update.installerName)`. Within the function it is passed into `join`, `test`; assigned or updated later in the function.

installerPath: installerPath stores a resolved filesystem or route value. It is initialized from `path.join(updateRoot, installerName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `emit`, `powershellSingleQuoted`, `writeFile`; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch(update.installerUrl, {`. Within the function it is accesses properties/methods `arrayBuffer`, `ok`, `status`; passed into `Error`, `from`; assigned or updated later in the function.

installerBuffer: installerBuffer stores the completed result of an awaited operation. It is initialized from `Buffer.from(await response.arrayBuffer())`. Within the function it is accesses properties/methods `length`; passed into `writeFile`; assigned or updated later in the function.

updateLogPath: updateLogPath stores a resolved filesystem or route value. It is initialized from `path.join(updateRoot, `update-${safeUpdateFileSegment(update.latestVersion)}.log`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `emit`, `powershellSingleQuoted`; assigned or updated later in the function.

currentExecutable: currentExecutable stores a resolved filesystem or route value. It is initialized from /OMB Portfolio Builder\exe\$/i.test(process.execPath || "") ? process.execPath : "". Within the function it is passed into dirname, from; assigned or updated later in the function.

currentInstallDirectory: currentInstallDirectory stores a resolved filesystem or route value. It is initialized from currentExecutable ? path.dirname(currentExecutable) : "". It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into from; assigned or updated later in the function.

localAppDataExecutable: localAppDataExecutable stores a environment-derived value for temporary calculation. It is initialized from process.env.LOCALAPPDATA. Within the function it is passed into from; assigned or updated later in the function.

legacyManagedApplicationExecutable: legacyManagedApplicationExecutable stores a resolved filesystem or route value. It is initialized from path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder", "OMB Portfolio Builder.exe"). Within the function it is assigned or updated later in the function.

legacyFlatApplicationExecutable: legacyFlatApplicationExecutable stores a resolved filesystem or route value. It is initialized from path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder.exe"). Within the function it is assigned or updated later in the function.

relaunchCandidates: relaunchCandidates stores a computed value for lookup collection. It is initialized from Array.from(new Set([. Within the function it is iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

relaunchCandidatesPs: relaunchCandidatesPs stores a string value for lookup collection. It is initialized from `@(\${relaunchCandidates.map(powershellSingleQuoted).join(", ")}`. Within the function it is assigned or updated later in the function.

launcherPath: launcherPath stores a resolved filesystem or route value. It is initialized from path.join(updateRoot, `run-update-\${safeUpdateFileSegment(update.latestVersion)}.ps1`). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into emit, writeFile; assigned or updated later in the function.

launcherCommandPath: launcherCommandPath stores a resolved filesystem or route value. It is initialized from path.join(updateRoot, `run-update-\${safeUpdateFileSegment(update.latestVersion)}.cmd`). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into spawn, writeFile; assigned or updated later in the function.

launcherScript: launcherScript stores a list for temporary calculation. It is initialized from [. Within the function it is passed into writeFile; assigned or updated later in the function.

launcherCommandScript: launcherCommandScript stores a list for temporary calculation. It is initialized from [. Within the function it is passed into writeFile; assigned or updated later in the function.

child: child stores a resolved filesystem or route value. It is initialized from spawn("cmd.exe", ["/d", "/c", launcherCommandPath], {. Within the function it is accesses properties/methods unref; assigned or updated later in the function.